



Latest updates: <https://dl.acm.org/doi/10.1145/3698803>

Published: 20 December 2024

RESEARCH-ARTICLE

[Citation in BibTeX format](#)

Common Neighborhood Estimation over Bipartite Graphs under Local Differential Privacy

YIZHANG HE, UNSW Sydney, Sydney, NSW, Australia

KAI WANG, Antai College of Economics and Management, Shanghai, China

WENJIE ZHANG, UNSW Sydney, Sydney, NSW, Australia

XUEMIN LIN, Antai College of Economics and Management, Shanghai, China

YING ZHANG, University of Technology Sydney, Sydney, NSW, Australia

Open Access Support provided by:

UNSW Sydney

University of Technology Sydney

Antai College of Economics and Management

Common Neighborhood Estimation over Bipartite Graphs under Local Differential Privacy

YIZHANG HE, University of New South Wales, Australia

KAI WANG, Antai College of Economics and Management, Shanghai Jiao Tong University, China

WENJIE ZHANG, University of New South Wales, Australia

XUEMIN LIN, Antai College of Economics and Management, Shanghai Jiao Tong University, China

YING ZHANG, University of Technology Sydney, Australia

Bipartite graphs, formed by two vertex layers, arise as a natural fit for modeling the relationships between two groups of entities. In bipartite graphs, common neighborhood computation between two vertices on the same vertex layer is a basic operator, which is easily solvable in general settings. However, it inevitably involves releasing the neighborhood information of vertices, posing a significant privacy risk for users in real-world applications. To protect edge privacy in bipartite graphs, in this paper, we study the problem of estimating the number of common neighbors of two vertices on the same layer under edge local differential privacy (edge LDP). The problem is challenging in the context of edge LDP since each vertex on the opposite layer of the query vertices can potentially be a common neighbor. To obtain efficient and accurate estimates, we propose a multiple-round framework that significantly reduces the candidate pool of common neighbors and enables the query vertices to construct unbiased estimators locally. Furthermore, we improve data utility by incorporating the estimators built from the neighbors of both query vertices and devise privacy budget allocation optimizations. These improve the estimator's robustness and consistency, particularly against query vertices with imbalanced degrees. Extensive experiments on 15 datasets validate the effectiveness and efficiency of our proposed techniques.

CCS Concepts: • **Networks** → **Network privacy and anonymity**; • **Information systems** → **Network data models**.

Additional Key Words and Phrases: Motif Counting, Differential Privacy

ACM Reference Format:

Yizhang He, Kai Wang, Wenjie Zhang, Xuemin Lin, and Ying Zhang. 2024. Common Neighborhood Estimation over Bipartite Graphs under Local Differential Privacy. *Proc. ACM Manag. Data* 2, 6 (SIGMOD), Article 228 (December 2024), 26 pages. <https://doi.org/10.1145/3698803>

1 Introduction

Bipartite graphs have been widely used to represent connections between two sets of entities. Examples of real-world bipartite graphs include user-item networks in E-commerce [25, 44], people-location networks in contact tracing [4], and user-page networks in social analysis [34]. In bipartite graphs, finding the common neighbors of two vertices is a basic operation in many tasks. For

Authors' Contact Information: Yizhang He, University of New South Wales, Australia, yizhang.he@unsw.edu.au; Kai Wang, Antai College of Economics and Management, Shanghai Jiao Tong University, China, w.kai@sjtu.edu.cn; Wenjie Zhang, University of New South Wales, Australia, wenjie.zhang@unsw.edu.au; Xuemin Lin, Antai College of Economics and Management, Shanghai Jiao Tong University, China, xuemin.lin@sjtu.edu.cn; Ying Zhang, University of Technology Sydney, Australia, ying.zhang@uts.edu.au.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2836-6573/2024/12-ART228

<https://doi.org/10.1145/3698803>

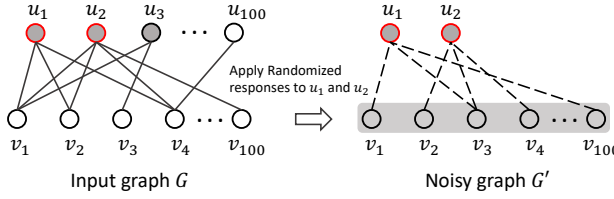


Fig. 1. A bipartite graph and its corresponding noisy graph by applying randomized responses to u_1 and u_2 .

example, the similarity between two vertices can be computed using Jaccard similarity, which is the ratio of the number of their common neighbors to their combined unique neighbors [24, 43, 55]. Additionally, common neighbor counts can help prune unpromising vertices in (p, q) -biclique counting [53, 57]. Other tasks that benefit from counting common neighbors in bipartite graphs include anomaly detection [42], bipartite graph projection [39, 59], bipartite clustering coefficient computation [2, 17], community search [1, 9, 45], and wedge-based motif counting [46, 51].

Although computing common neighbors is straightforward in the conventional setting, it inevitably involves releasing the neighborhood information of the vertices, posing a significant privacy risk for users in real-world applications. For instance, in user-item networks, disclosing identical items in the shopping carts of two users in online shopping platforms (e.g., eBay and Amazon) significantly compromises users' privacy. Hence, it is crucial to estimate the common neighborhood in a privacy-preserving manner, which remains an unresolved research gap.

In the literature, *differential privacy* (DP) [10] has become the gold standard for privacy-preserving computation, which provides a mathematical framework to quantify permissible privacy loss. Among the various DP models, *edge local differential privacy* (edge LDP) [35, 56, 60] has been widely adopted to protect the user's private connections. Edge LDP is a robust privacy protection protocol that requires each vertex to perturb its local data (e.g., degrees and neighbors) before transmitting it to the data curator.

In this paper, we study the problem of privacy-preserving common neighborhood estimation over bipartite graphs. Specifically, given a bipartite graph G and two query vertices u and w in G on the same vertex layer with edge LDP. A random algorithm satisfies edge LDP when the probabilities of observing its output from any two vertices, whose neighbor sets differ by one vertex, are indistinguishable within a factor of e^ϵ . This ensures deniability for the existence of the edge (u, v) . In this context, ϵ represents the privacy budget, determining the acceptable level of privacy loss. Clients can adjust this parameter to tradeoff between privacy and data utility. In addition, common neighbor counting under edge LDP is the first step in addressing other problems under edge LDP, such as vertex similarity computation [24] and (p, q) -biclique counting [52].

Challenges. In this paper, we aim to design accurate and efficient common neighbor estimation algorithms on bipartite graphs with edge LDP. We face the following three main challenges.

- **Challenge 1:** In the literature, Randomized Response [47] is widely employed to achieve edge LDP, flipping the entries of the adjacency matrix between “0” and “1” to construct a noisy graph. However, counting common neighbors on the noisy graph results in severe overcounting and bias, because the noisy graph is generally much denser. For instance, we examine the performance of this naive approach (Naive) across 1000 runs for a pair of query vertices on the dataset *rmwiki* in Fig. 2. The blue distribution, representing the estimates of Naive, deviates significantly to the right from the true count (indicated by the black dashed line). This substantial shift highlights the difficulty in accurately estimating the number of common neighbors using the noisy graph.

- **Challenge 2:** Due to the constraints of edge LDP, we have to start with all vertices on the opposite layer of the query vertices as a candidate pool to estimate the common neighbors. This involves potentially $O(n)$ independent random variables, leading to a substantial margin of error. Reducing this candidate pool and developing an unbiased estimator that relies on fewer random variables for enhanced data utility presents a challenging task.

- **Challenge 3:** When the vertices and the data curator can interact for multiple rounds, the privacy budget must be divided among each round. However, allocating more budget to one round reduces the budget available for others. In addition, different pairs of query vertices likely need to be handled differently. Thus, finding the optimal allocation of privacy budgets to different rounds based on the query vertices requires special attention.

Our approaches. To address Challenge 1, we propose a one-round algorithm OneR that obtains unbiased estimates of common neighbors by leveraging the probability at which the entries in the adjacency matrix are flipped to compensate for over-counting. Specifically, for each query vertex, OneR applies randomized responses to both query vertices to generate noisy edges. For every vertex v on the opposite layer of the query vertices, we estimate whether v is a common neighbor of u and w . Then OneR aggregates these estimates to obtain an unbiased estimate. While OneR addresses the challenge of the dense noisy graph and generates unbiased estimates, it relies on a large pool of candidates, compromising the utility of the data. As shown in Fig. 2, the yellow distribution representing the estimates of OneR appears symmetrical around the true common neighbor count with fat tails on both sides. This implies that the estimates are unbiased but have high variance.

To address Challenge 2, we propose a multiple round framework, allowing the query vertices to download the noisy edges from the previous round and reduce the candidate pool to their neighbors. Under this framework, we devise a single-source algorithm MultiR-SS, which returns an unbiased estimator for the number of common neighbors by leveraging the neighborhood of u . Specifically, in the first round, MultiR-SS applies randomized responses to vertex w to generate noisy edges. In the second round, the vertex u retrieves these noisy edges from the data curator and then constructs a single-source estimator using its neighbors in the original graph and the neighbors of w in the noisy graph. To comply with edge LDP, the Laplace mechanism [10] is used to add noise to the single-source estimator before it is released to the data curator. This noise is scaled with the global sensitivity, defined as the maximum difference observed in the single-source estimator between two vertices whose neighbors differ by one edge. MultiR-SS achieves significantly better data utility compared to OneR by employing a multiple-round framework that reduces the candidate pool of common neighbors to the neighborhoods of the query vertices. As shown in Fig. 2, the green distribution represents the estimates of MultiR-SS, which preserves the unbiasedness and is more concentrated around the true value compared to OneR.

To tackle Challenge 3, we propose a double-source algorithm MultiR-DS under the multiple-round framework, which integrates two single-source estimators via a weighted average. In addition, we propose novel privacy budget allocation optimizations that allow MultiR-DS to dynamically adjust its privacy budgets for different rounds and the contribution of each single-source estimator for minimized L2 loss. Specifically, when the incoming query vertices have large degrees, MultiR-DS tends to devote more privacy budget to noisy graph construction. When the query vertices have very imbalanced degrees, MultiR-DS will favor the single-source estimator associated with the low-degree vertex, which depends on fewer random variables and induces less variance. In doing so, MultiR-DS further reduces L2 loss compared to MultiR-SS and is more robust to query vertices with high degrees or unbalanced degrees. In Fig. 2, the red distribution depicts the estimates of MultiR-DS. Here the degrees of the query vertices are highly imbalanced (556 and 2). In this case, MultiR-DS yields unbiased and more concentrated estimates compared to MultiR-SS.

Contributions. Here we summarize our principal contributions.

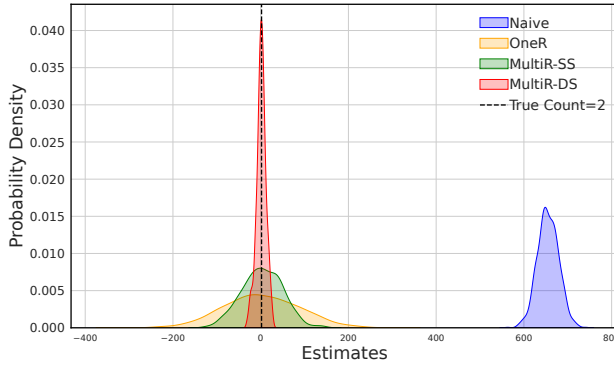


Fig. 2. The estimate distribution on rmwiki when $\epsilon = 1$.

- To the best of our knowledge, we are the first to study accurate and efficient common neighborhood estimation on bipartite graphs under edge LDP.
- To address the over-counting issue with the Naive algorithm, we propose a one-round algorithm OneR to return unbiased estimates, which leverages the probabilistic nature of the noisy graph to compensate for over-counting.
- We propose a multiple-round framework and devise a single-source algorithm (MultiR-SS), which allows one query vertex to download the noisy edges from the other query vertex and construct an unbiased estimator locally. This drastically reduces L2 loss because the search scope for the common neighbors is reduced to the neighborhood of one query vertex.
- Under the multiple-round framework, we propose a double-source algorithm (MultiR-DS) that constructs two unbiased estimators for each query vertex and combines them through a weighted average. MultiR-DS further reduces L2 loss by dynamically adjusting the allocation of the privacy budget and the contribution of two estimators based on the incoming query vertices.
- We conduct extensive experiments on 15 real-world datasets to evaluate the proposed algorithms. The multiple-round algorithms MultiR-SS and MultiR-DS produce significantly smaller mean relative errors than Naive and OneR across all datasets. MultiR-DS is especially robust to query vertices with imbalanced degrees.

2 Preliminary

Table 1. Summary of Notations

Notation	Definition
G	a bipartite graph
$\mathcal{A}$	the adjacency matrix
$\mathcal{A}_u$	the neighbor list of the vertex u
$N(u, G)$	the neighbors of u in G
$\deg(u, G)$	the degree of u in G
ϵ	a privacy budget
G'	a noisy bipartite graph
$C_2(u, w)$	the number of common neighbors between u and w
Δg	the global sensitivity of a function g
$\tilde{f}_u$	the single-source estimator based on $N(u, G)$
$\tilde{f}_w$	the single-source estimator based on $N(w, G)$
f^*	the double source estimator based on both $\tilde{f}_u$ and $\tilde{f}_w$

2.1 Problem definition

We consider an unweighted bipartite graph $G(V = (U, L), E)$. $V = U \cup L$ denotes the set of vertices, where U and L represent the upper and lower layer, respectively. The vertices in U and L are called the upper vertices and the lower vertices. $E \subseteq U \times L$ denotes the set of edges. We use $n_1 = |U(G)|$ and $n_2 = |L(G)|$ to denote the number of upper and lower vertices, respectively, and $m = |E|$ to represent the number of edges in G . The adjacency matrix $\mathcal{A}$ for G is of dimensions $n \times n$, where $\mathcal{A}[u, v] = 1$ if there exists an edge between the vertices u and v and 0 otherwise. The u -th row of $\mathcal{A}$ (including both “1” and “0”) is the *neighbor list* of u , denoted by $\mathcal{A}_u$. In addition, the set of neighbors of a vertex u in G is denoted by $N(u, G)$, and its degree is denoted by $\deg(u, G) = |N(u, G)|$. We use $d_{\max}(U)$ and $d_{\max}(L)$ to represent the maximum degree among the upper vertices and lower vertices, respectively.

DEFINITION 1. Common neighbors. Let u and w be two vertices on the same layer of a bipartite graph G . The common neighbors of u and w are the vertices adjacent to both u and w in G , i.e., $N(u, G) \cap N(w, G)$. Here $N(u, G)$ represents the set of neighbors of vertex u in graph G . We use $C_2(u, w)$ to denote the number of common neighbors of u and w , i.e., $C_2(u, w) = |N(u, G) \cap N(w, G)|$.

Classic differential privacy (DP) operates under a central model, where a central data curator manages a dataset D [10]. D and D' are neighboring datasets if they differ by one data record. DP ensures that query outputs on these neighboring datasets are hard to distinguish. When extending DP under the central model to protect edge privacy in graphs, neighboring datasets refer to two graphs that vary by a single edge. However, the assumption with the central model that the data curator with access to the entire graph can be trusted can be impractical in real-world scenarios. Thus, we adopt the widely-used ϵ -edge local differential privacy (edge LDP), which enables each vertex to locally perturb its data before transmission to the data curator [36, 56, 60]. Under edge LDP, the neighboring datasets are two neighbor lists differing by one bit.

DEFINITION 2. ϵ -edge local differential privacy. Let G be a bipartite graph and $\epsilon > 0$. For each vertex $u \in V(G)$, let R_u with domain $\{0, 1\}^n$ be a randomized algorithm of vertex u . R_u provides ϵ -edge LDP if for any two neighbor lists $\mathcal{A}_u, \mathcal{A}'_u$ that differ in one bit and any $S \subseteq \text{Range}(R_u)$:

$$\Pr(R_u(\mathcal{A}_u) \in S) \leq e^\epsilon \Pr(R_u(\mathcal{A}'_u) \in S)$$

Here ϵ is called the privacy budget.

Problem Statement. Given a bipartite graph G , a privacy budget ϵ , and a pair of vertices u and w on the same layer of G , the *common neighborhood estimation* problem aims to estimate $C_2(u, w)$ while satisfying ϵ -edge LDP.

Without loss of generality, we assume that u and w are in the lower layer of G . In this paper, we use the *expected L2 loss* to evaluate the quality of estimates for the number of common neighbors.

DEFINITION 3. Expected L2 loss. Given two vertices u and w on a bipartite graph G , and an estimate f for the number of common neighbors between u and w , the *expected L2 loss* of f is the expected squared error between $C_2(u, w)$ and f , i.e.,

$$l2(C_2(u, w), f) = \mathbb{E}((C_2(u, w) - f)^2)$$

2.2 Warm up

ϵ -edge LDP ensures that if two vertices have neighbor lists differing by just one bit, they cannot be reliably distinguished based on the outputs of the randomized algorithms. In this part, we introduce the most common methods for providing edge LDP, which are randomized responses and the

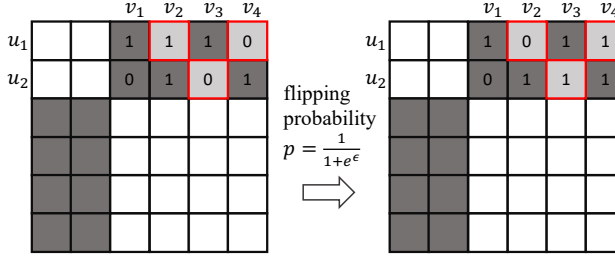


Fig. 3. randomized responses on a bipartite graph.

Laplace mechanism. Then, we present a naive solution to the common neighborhood estimation problem.

Warner's randomized response. One effective method for achieving ϵ -edge LDP is through randomized responses (RR), initially introduced as a survey technique to allow confidential answers to sensitive inquiries such as criminal or sexual activities [47]. In essence, participants are asked to answer the questions honestly with probability. This concept has been adapted for graph applications to ensure edge local differential privacy [18, 35]. Specifically, each entry $x \in 0, 1$ of a neighbor list is perturbed with a probability of $\frac{1}{1+e^\epsilon}$, where ϵ denotes the privacy budget.

$$RR(x) = \begin{cases} 1-x & \text{with probability } \frac{1}{1+e^\epsilon} \\ x & \text{with probability } \frac{e^\epsilon}{1+e^\epsilon} \end{cases}$$

Current state-of-the-art methods on general graphs perturb each entry in the lower triangle of the adjacency matrix [18, 19]. However, to ensure that the resulting noisy graph G' is bipartite, we only perturb entries in $\mathcal{A}$ that represent potential edges in a bipartite graph. Without loss of generality, we assume that the upper vertices have smaller IDs than the lower vertices. In this way, the adjacency matrix $\mathcal{A}$ can be divided into 4 blocks, where the ones on the diagonal are empty because the vertices of the same layer cannot be connected. In this paper, we only apply randomized responses to the neighbor lists of the query vertices u and w . When $u, w \in U(G)$, we flip the entries in $\mathcal{A}_u \cap L(G)$ and $\mathcal{A}_w \cap L(G)$. When $u, w \in L(G)$, we flip the entries in $\mathcal{A}_u \cap U(G)$ and $\mathcal{A}_w \cap U(G)$. In doing so, we avoid generating noisy edges not allowed in bipartite graphs. We denote the noisy graph with respect to a privacy budget ϵ by G'_ϵ .

EXAMPLE 1. We illustrate applying randomized responses to a bipartite graph with two upper vertices and four lower vertices in Fig. 3. The left shows the original adjacency matrix and the right side shows the matrix after applying randomized responses. Rows and columns are ordered with upper vertices preceding lower vertices. White squares indicate zeros, as edges within the same layer are not allowed in bipartite graphs. To find common neighbors of the upper vertices u and w , we apply randomized responses to their neighbor lists, which affect the upper right block with eight grey squares. With the privacy budget ϵ , each grey block is flipped with probability $\frac{1}{1+e^\epsilon}$. Light grey squares outlined in red represent flipped entries.

Calibrating noise with global sensitivity. To achieve ϵ -edge LDP, it is necessary to add noise to any data transmitted from a vertex to the data curator. The Laplace mechanism is used for this purpose, which calibrates the amount of noise with the *global sensitivity* of the transmitted data.

DEFINITION 4. Global sensitivity. Consider a bipartite graph G . Let $\mathcal{A}_u$ be the neighbor list of vertex u . Let $\mathcal{A}'_u$ be a neighbor list that differs from $\mathcal{A}_u$ in at most one entry. The global sensitivity of a function $f : \mathcal{A}_u \rightarrow \mathbb{R}$ is:

$$\Delta f = \max_{\mathcal{A}_u, \mathcal{A}'_u} |f(\mathcal{A}_u) - f(\mathcal{A}'_u)|$$

DEFINITION 5. The Laplace Mechanism. Given a privacy budget ϵ and any function f , the Laplace mechanism is defined as:

$$\tilde{f} = f + \text{Lap}\left(\frac{\Delta f}{\epsilon}\right)$$

Here $\tilde{f}$ is the noisy version of f and $\text{Lap}(\cdot)$ is the probability density function of the Laplace distribution.

By applying the Laplace mechanism, we allow the vertices to send local graph statistics with calibrated noise to the data curator while satisfying ϵ -edge LDP.

Edge LDP can also be obtained via composition (i.e., combine multiple edge LDP algorithms). For instance, *sequential composition* [21, 35] enables the sequential application of multiple edge LDP algorithms (M_i), each consuming some privacy budgets (ϵ_i), and ensures that the overall process satisfies $\sum_i \epsilon_i$ -edge LDP. *Parallel composition* states that if disjoint subsets of the graph are processed by different edge LDP algorithms (M_i) with privacy budget ϵ_i , then the overall mechanism running these algorithms in parallel satisfies $\max_i(\epsilon_i)$ -edge LDP [54]. Furthermore, edge LDP is immune to *post-processing* [18, 54], allowing the data curator to apply any post-processing to the graph statistics received from the vertices without compromising the privacy guarantees.

Algorithm 1: Naive

Input: G : a bipartite graph; ϵ : a privacy budget; u, w : two query vertices from the same layer

Output: $\tilde{f}_1$: the naive estimator of $C_2(u, w)$

// **vertex side:**

1 **foreach** $i \in \{u, w\}$ **do**

2 **foreach** $j \in \text{the opposite layer from } u \text{ and } w$ **do**

3 perturb $\mathcal{A}'[i, j] \leftarrow \begin{cases} 1 - \mathcal{A}[i, j], & \text{w.p. } \frac{1}{1+e^\epsilon} \\ \mathcal{A}[i, j] & \text{w.p. } \frac{e^\epsilon}{1+e^\epsilon} \end{cases}$

4 send noisy edges to the data curator;

5 $G'_\epsilon \leftarrow$ the noisy graph constructed from $\mathcal{A}'[i, j]$;

// **curator side:**

6 $\tilde{f}_1 \leftarrow |N(u, G'_\epsilon) \cap N(w, G'_\epsilon)|$;

7 **return** $\tilde{f}_1$;

A naive approach. Given that randomized responses preserve ϵ -edge LDP, a naive approach is to count the number of common neighbors of u and w on the noisy graph constructed by randomized responses, as outlined in Algorithm 1. Note that for our problem, we only need to apply randomized responses to u and w . Specifically, given a privacy budget $\epsilon > 0$, Naive flips the entries $\mathcal{A}[i, j]$ with a probability $\frac{1}{1+e^\epsilon}$, where $i \in \{u, w\}$ (Lines 1-4). The data curator collects the noisy edges from u and w and constructs a noisy graph G'_ϵ (Line 5). In this way, we do not need to analyze the global sensitivity for Naive because it does not involve the Laplace mechanism and only relies on randomized responses to provide edge LDP. Then, the naive estimator $\tilde{f}_1$ is calculated taking the intersection of the neighbors of u and w in G'_ϵ , i.e., $\tilde{f}_1 = |N(u, G'_\epsilon) \cap N(w, G'_\epsilon)|$ (Line 6). However, since there are generally more “0”s than “1”s in the neighbor lists of u and w , applying randomized responses usually results in a much denser noisy graph G'_ϵ , which results in severe overcounting $C_2(u, w)$.

Theoretical analysis for Naive. Without loss of generality, we assume that the query vertices u and w are from the lower layer $L(G)$ when analyzing the time costs, communication costs, and expected L2 losses. The time costs are divided between the vertex side and the curator side. On the vertex side, the time costs incurred by randomized responses is $O(n_1)$, where n_1 is the number

of vertices in $U(G)$. On the curator side, the dominating cost is incurred by intersecting $N(u, G'_\epsilon)$ and $N(w, G'_\epsilon)$, which takes $O(\min(\deg(u, G'_\epsilon), \deg(w, G'_\epsilon)))$. Thus, the overall time complexity is $O(n_1)$. The communication costs are incurred only during randomized responses, where vertices u and w send noisy edges to the data curator. For vertex u , the expected number of noisy edges is $d_u \times (1 - p) + (n_1 - d_u) \times p$, where $p = \frac{1}{1+e^\epsilon}$. Similarly, for vertex w , the expected number of noisy edges is $d_w \times (1 - p) + (n_1 - d_w) \times p$. Thus, the overall communication cost is $O\left(\frac{e^\epsilon - 1}{e^\epsilon + 1}(d_u + d_w) + \frac{2n_1}{1+e^\epsilon}\right)$. In the following, we analyze the expected L2 loss of the estimator $\tilde{f}_1$ returned by Naive.

THEOREM 1. *Given a bipartite graph G , a privacy budget ϵ , and a pair of query vertices u and w in $L(G)$, the expected L2-loss of the estimator for $\tilde{f}_1$ in Algorithm 1 is $O(\frac{n_1^2}{(1+e^\epsilon)^4})$. Here, n_1 represents the number of vertices in $U(G)$.*

PROOF. The naive estimator $\tilde{f}_1 = |N(u, G'_\epsilon) \cap N(w, G'_\epsilon)|$. Let $p = \frac{1}{1+e^\epsilon}$ be the flipping probability during the randomized responses. Note that each entry $\mathcal{A}'[i, j]$ on the adjacency matrix of the noisy graph follows a Bernoulli distribution with a parameter p (if $\mathcal{A}[i, j] = 0$) or $1 - p$ (if $\mathcal{A}[i, j] = 1$). Since $\tilde{f}_1 \geq 0$, we have the following inequality:

$$l_2(\tilde{f}_1, C_2(u, w)) = \mathbb{E}(\tilde{f}_1^2) - 2C_2(u, w)\mathbb{E}(\tilde{f}_1) + C_2(u, w)^2 = O(\mathbb{E}(\tilde{f}_1^2))$$

Since $\tilde{f}_1 = \sum_{v \in U(G)} \mathcal{A}'[u, v]\mathcal{A}'[v, w]$, we have:

$$\begin{aligned} \mathbb{E}(\tilde{f}_1^2) &= \sum_{v \in U(G)} \mathbb{E}((\mathcal{A}'[u, v]\mathcal{A}'[v, w])^2) \\ &\quad + 2 \sum_{v_i < v_j \in U(G)} \mathbb{E}(\mathcal{A}'[u, v_i]\mathcal{A}'[v_i, w]\mathcal{A}'[u, v_j]\mathcal{A}'[v_j, w]) \\ &\leq O\left(\binom{n_1}{2}(1-p)^4\right) = O(n_1^2(1-p)^4) = O\left(\frac{n_1^2 e^{4\epsilon}}{(1+e^\epsilon)^4}\right) \end{aligned}$$

The last step is due to $p = \frac{1}{1+e^\epsilon}$. □

THEOREM 2. *The Naive algorithm satisfies ϵ -edge LDP.*

PROOF. Since the randomized responses provide ϵ -edge LDP [18, 20], Lines 1-4 of the algorithm satisfy ϵ -edge LDP. In addition, edge LDP is immune to post-processing, which means that any analysis (Lines 5-6) conducted on the noisy graph preserves edge LDP. Thus, the theorem holds. □

3 A one-round approach

To address the problem of overcounting with the Naive approach, we propose a one-round algorithm OneR, which exploits the probabilistic nature of the noisy graph to obtain unbiased estimates of $C_2(u, w)$. Specifically, OneR leverages the flipping probability during randomized responses to construct an unbiased estimator for each vertex on the opposite layer of the query vertices. In doing so, we do not need to analyze the global sensitivity for OneR, as it only relies on randomized responses to ensure edge LDP and does not involve the Laplace mechanism. Then, OneR aggregates these estimates to obtain unbiased counts of common neighbors. First, we investigate the unbiased estimator of $\mathcal{A}[i, j]$ for two vertices i and j in the bipartite graph.

3.1 An unbiased estimator for $\mathcal{A}[i, j]$

Consider a bipartite graph G . Let ε be the privacy budget. We use G'_ε to represent the noisy graph from applying randomized responses to the edges in G . During randomized responses, each entry in the neighbor list is flipped with a probability $p = \frac{1}{1+e^\varepsilon}$. Note that each entry $\mathcal{A}'[i, j]$ on the adjacency matrix of the noisy graph follows a Bernoulli distribution with a parameter p (when $\mathcal{A}[i, j] = 0$) or $1 - p$ (when $\mathcal{A}[i, j] = 1$). Based on this, we have the following equations which link the expected value of $\mathcal{A}[i, j]'$ and $\mathcal{A}[i, j]$:

$$\mathbb{E}(\mathcal{A}'[i, j]) = \begin{cases} p, & \text{if } \mathcal{A}[i, j] = 0 \\ 1 - p & \text{if } \mathcal{A}[i, j] = 1 \end{cases}$$

This can be rearranged into one single equation: $\mathbb{E}(\mathcal{A}'[i, j]) = \mathcal{A}[i, j] + p(1 - 2\mathcal{A}[i, j])$. Solving this equation for $\mathcal{A}[i, j]$ leads to:

$$E\left(\frac{\mathcal{A}'[i, j] - p}{1 - 2p}\right) = \mathcal{A}[i, j]$$

Let $\phi(i, j) = \frac{\mathcal{A}'[i, j] - p}{1 - 2p}$. It immediately follows that $\phi(i, j)$ is an unbiased estimator of $\mathcal{A}[i, j]$. We can also analyze the variance of $\phi(i, j)$:

$$\text{Var}(\phi(i, j)) = \text{Var}\left(\frac{\mathcal{A}'[i, j] - p}{1 - 2p}\right) = \frac{p(1 - p)}{(1 - 2p)^2} \quad (1)$$

The last step is because $\mathcal{A}'[i, j]$ is a Bernoulli variable with a probability p or $1 - p$, which leads to a variance of $p(1 - p)$.

Algorithm 2: OneR

Input: G : a bipartite graph; ε : a privacy budget; u, w : two query vertices from the same layer

Output: $\tilde{f}_2$: the one-round unbiased estimator for $C_2(u, w)$

```

1 foreach  $i \in \{u, w\}$  do
2   foreach  $j \in \text{the opposite layer from } u \text{ and } w$  do
3      $\text{perturb } \mathcal{A}'[i, j] \leftarrow \begin{cases} 1 - \mathcal{A}[i, j], & \text{w.p. } \frac{1}{1+e^\varepsilon} \\ \mathcal{A}[i, j] & \text{w.p. } \frac{e^\varepsilon}{1+e^\varepsilon} \end{cases}$ 
4   send noisy edges to the data curator;
5    $G'_\varepsilon \leftarrow \text{the noisy graph constructed from } \mathcal{A}'[i, j]$ ;
6    $\tilde{f}_2 \leftarrow 0$ ;  $p \leftarrow \frac{1}{1+e^\varepsilon}$ ;
7   foreach  $v \in V(G'_\varepsilon)$  on the opposite layer as  $u$  and  $w$  do
8      $\tilde{f}_2 \leftarrow \tilde{f}_2 + (\mathcal{A}'[u, v] - p)(\mathcal{A}'[v, w] - p)/(1 - 2p)^2$ ;
9 return  $\tilde{f}_2$ ;

```

3.2 An unbiased estimator for $C_2(u, w)$

In this part, we derive an unbiased estimator for $C_2(u, w)$ based on the noisy graph from randomized responses. Without loss of generality, let's assume that both u and w are in $L(G)$. By definition, $C_2(u, w) = |N(u, G) \cap N(w, G)| = \sum_{v \in U(G)} \mathcal{A}[u, v] \mathcal{A}[v, w]$. This implies that we need to estimate $\mathcal{A}[u, v] \mathcal{A}[v, w]$ for all $v \in U(G)$ in an unbiased way. Since $\mathcal{A}'[u, v]$ and $\mathcal{A}'[v, w]$ are independent of each other, we have $\mathbb{E}(\phi(u, v)\phi(v, w)) = \mathcal{A}[u, v] \mathcal{A}[v, w]$, which leads to the following estimator for $C_2(u, w)$.

THEOREM 3. Consider a bipartite graph G , a privacy budget ϵ , and two vertices u and w in $L(G)$. Let G'_ϵ be the noisy graph after applying the randomized response to G w.r.t. ϵ . Let $p = \frac{1}{1+e^\epsilon}$ be the flipping probability. We have $\mathbb{E}(\tilde{f}_2(u, w)) = C_2(u, w)$ where

$$\tilde{f}_2(u, w) = \sum_{v \in U(G)} \frac{(\mathcal{A}'[u, v] - p)(\mathcal{A}'[v, w] - p)}{(1 - 2p)^2} \quad (2)$$

is an unbiased estimate for $C_2(u, w)$.

The proof of Theorem 3 involves summing $\phi(u, v)\phi(v, w)$ over all $v \in U(G)$. In particular, when u and w belong to $U(G)$, similar results are derived with v in $L(G)$. Based on Theorem 3, we design a one-round algorithm OneR, which estimates $C_2(u, w)$ based on the noisy graph generated from randomized responses, as outlined in Algorithm 2. In Lines 1-6, OneR constructs a noisy graph G'_ϵ by applying randomized responses to u and w . Then, it computes $\tilde{f}_2(u, w)$ by considering all vertices on the opposite layer (i.e., $U(G)$) as candidates for the common neighbors of u and w (Lines 7-8). In practice, to efficiently compute $\tilde{f}_2(u, w)$ and avoid visiting all candidate vertices, the expression in Equation 2 can be expanded as such.

$$\tilde{f}_2(u, w) = N_1 \frac{(1-p)^2}{(1-2p)^2} - (N_2 - N_1) \frac{(1-p)p}{(1-2p)^2} + (n_1 - N_2) \frac{p^2}{(1-2p)^2}$$

Here N_1 denotes the number of common neighbors of u and w in the noisy graph G'_ϵ . N_2 denotes size of the union of the neighbor sets of u and w in G'_ϵ . n_1 represents the number of vertices in $|U(G)|$ (i.e., the opposite layer of u and w). In this way, we only need to compute the intersection and union of the neighbor sets of u and w in G'_ϵ to efficiently obtain $\tilde{f}_2(u, w)$.

EXAMPLE 2. We illustrate the construction of the unbiased estimator $\tilde{f}_2$ returned by the OneR algorithm using the bipartite graph shown in Fig.1, focusing on u_1 and u_2 as query vertices with three common neighbors. The outline of the query vertices is highlighted in red. On the right side of Fig.1, we present the noisy graph constructed by applying randomized responses to u_1 and u_2 . Note that we only need to apply randomized responses to u_1 and u_2 . The dashed lines in the graph represent the resulting noisy edges. The vertices shaded in grey depict the candidate pool for common neighbors between u_1 and u_2 in OneR, including all vertices on the opposite layer from u_1 and u_2 . According to Equation 2, $\tilde{f}_2$ relies on $n_2 = |L(G)| = 100$ random variables: $\mathcal{A}'[v_i, u_2]$, where $i \in [1, 100]$.

Theoretical analysis for OneR. Without loss of generality, we assume that the query vertices u and w are in $L(G)$ in the following analyses. On the vertex side, the time costs incurred by randomized responses is $O(n_1)$, where n_1 is the number of vertices in $U(G)$. On the curator side, the dominating cost is incurred by computing $\tilde{f}_2(u, w)$ in Lines 7-8, which can be implemented in $O(\deg(u, G'_\epsilon) + \deg(w, G'_\epsilon))$ time by computing the intersection and the union of $N(u, G'_\epsilon)$ and $N(w, G'_\epsilon)$. The overall time complexity is $O(n_1)$. The communication costs of OneR are incurred only during randomized responses, similar to Naive. The overall communication cost is $O\left(\frac{e^\epsilon - 1}{e^\epsilon + 1}(d_u + d_w) + \frac{2n_1}{1 + e^\epsilon}\right)$.

In the following, we analyze the expected L2 loss of $\tilde{f}_2$ returned by Algorithm 2 in Theorem 4.

THEOREM 4. The L2 loss of $\tilde{f}_2$ in Theorem 3 is $O\left(\frac{n_1 e^\epsilon}{(1 - e^\epsilon)^4}\right)$. Here, n_1 represents the number of vertices in $U(G)$ (i.e., the opposite layer to u and w).

PROOF. Since $\tilde{f}_2$ is unbiased, based on the bias-variance decomposition theorem [3], the L2 loss of f_1 equals its variance. Assume u and $w \in L(G)$.

$$\begin{aligned} l_2(\tilde{f}_2, C_2(u, w)) &= \text{Var}(\tilde{f}_2(u, w)) \\ &= \frac{1}{(1-2p)^4} \sum_{v \in U(G)} \text{Var}((\mathcal{A}'[u, v] - p)(\mathcal{A}'[v, w] - p)) \end{aligned}$$

Let $\eta = \mathcal{A}'[u, v] - p$ and $\theta = \mathcal{A}'[v, w] - p$. By construction, we know that $\mathbb{E}(\eta) = 1 - 2p$ when $v \in N(u, G)$ and $\mathbb{E}(\eta) = 0$ otherwise. Similarly, $\mathbb{E}(\theta) = 1 - 2p$ when $v \in N(w, G)$ and $\mathbb{E}(\theta) = 0$ otherwise. In addition, we have $\text{Var}(\eta) = \text{Var}(\theta) = p(1-p)$. This is because η and θ are shifted Bernoulli variables with the same variance.

$$\begin{aligned} \text{Var}(\tilde{f}_2(u, w)) &= \frac{1}{(1-2p)^4} \sum_{v \in U(G)} \text{Var}(\eta\theta) \\ &= \frac{p^2(1-p)^2}{(1-2p)^4} |U(G)| + \frac{p(1-p)}{(1-2p)^2} (\deg(u, G) + \deg(w, G)) \\ &= O\left(\frac{p^2(1-p)^2}{(1-2p)^4} |U(G)|\right) = O\left(\frac{n_1 e^{2\epsilon}}{(1-e^\epsilon)^4}\right) \end{aligned}$$

The last step is due to $p = \frac{1}{1+e^\epsilon}$. □

THEOREM 5. *The OneR algorithm satisfies ϵ -edge LDP.*

PROOF. Since the randomized responses provide ϵ -edge LDP [18, 20], Lines 1-4 of the algorithm satisfy ϵ -edge LDP. Additionally, edge LDP is immune to post-processing, meaning that any analysis conducted on the noisy graph (Lines 5-8) preserves edge LDP. Thus, the theorem holds. □

4 Multiple-round approaches

The OneR algorithm reflects our first attempt at obtaining an unbiased estimate of $C_2(u, w)$. However, as analyzed in Section 3, the L2 loss of OneR still contains a factor of n_1 , because OneR inevitably considers all potential vertices on the opposite layer from u and w as candidates for the common neighbors. To further improve data utility, in this section, we employ the classic multiple-round framework in the literature of graph analysis under edge LDP [18]. In the first round, we utilize a part of the privacy budget to construct a noisy graph via randomized responses. Then, we allow both u and w to download the noisy edges from each other and combine their local neighbors with the noisy edges to compute unbiased estimates of $C_2(u, w)$ locally. In the end, we use the remaining privacy budget to apply the Laplace mechanism to these unbiased estimates to comply with edge LDP. By adopting this multiple-round framework, we propose a Multiple-round Single Source algorithm (MultiR-SS) where we only rely on the local view of u to estimate $C_2(u, w)$. Then, we introduce the Multiple-round Double Source algorithm (MultiR-DS), which leverages the local neighborhoods of both u and w to optimize privacy budget allocation and balance the contribution of query vertices, resulting in minimized L2 loss.

4.1 A single-source estimator for $C_2(u, w)$

In this part, we introduce a two-round algorithm for estimating $C_2(u, w)$. First, ϵ_1 is utilized to construct a noisy graph by applying randomized responses to u and w . Then, the data curator releases the noisy graph. In the second round, vertex u integrates its local neighbors with the noisy

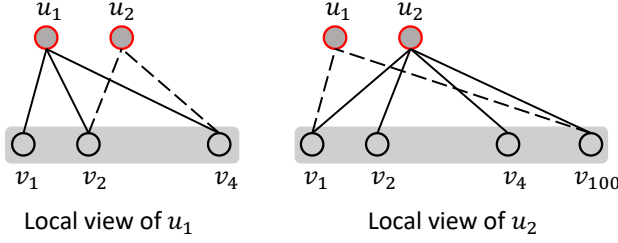


Fig. 4. The construction of $\tilde{f}_u$ and $\tilde{f}_w$ based on the local neighborhoods of u_1 and u_2 ($u = u_1, w = u_2$).

graph to derive an unbiased estimator for $C_2(u, w)$. Then, ε_2 is employed to apply the Laplace mechanism to add noise to this estimator.

Now we assume that the noisy graph G'_{ε_1} has already been constructed and discuss how to estimate $C_2(u, w)$ based on the local neighbors of u and the noisy neighbors of w . We start by noting that $C_2(u, w)$ can also be written as $\sum_{v \in N(u)} \mathcal{A}[v, w]$. Thus, when the neighbors of u are available, estimating $C_2(u, w)$ reduces to estimating $\mathcal{A}[v, w]$, which has already been addressed by $\phi(v, w) = \frac{\mathcal{A}'[v, w] - p}{1 - 2p}$ in Section 3. Here the flipping probability becomes $\frac{1}{1 + e^{\varepsilon_1}}$. Based on the above analysis, we derive the following unbiased estimate of $C_2(u, w)$ as

$$\begin{aligned} f_u(u, w) &= \sum_{v \in N(u, G)} \phi(v, w) = \sum_{v \in N(u, G)} \frac{\mathcal{A}'[v, w] - p}{1 - 2p} \\ &= |N(u, G) \cap N(w, G'_{\varepsilon_1})| \frac{1 - p}{1 - 2p} - |N(u, G) \setminus N(w, G'_{\varepsilon_1})| \frac{p}{1 - 2p} \end{aligned}$$

At this point, $f_u(u, w)$ is computed locally based on the neighborhood of u . To release it under edge LDP, we analyze the global sensitivity of $f_u(u, w)$ and apply the Laplace mechanism.

Global Sensitivity Analysis. By Definition 4, the global sensitivity of $f_u(u, w)$ is defined as $\Delta(f_u(u, w)) = \max_{u, u' \in V(G)} |f_u(u, w) - f_{u'}(u, w)|$, where u' represents a hypothetical vertex differing from u in its neighbor list at one entry. It follows:

$$\Delta(f_u(u, w)) \leq \max_{v'} |\phi(v', w)| = \frac{1 - p}{1 - 2p}$$

The last step is because the absolute value of $\phi(v', w)$ is either $\frac{1 - p}{1 - 2p}$ or $\frac{p}{1 - 2p}$. Since $p = \frac{1}{1 + e^{\varepsilon}} < \frac{1}{2}$, $\frac{1 - p}{1 - 2p}$ is always larger than $\frac{p}{1 - 2p}$. This bound suggests that we must add Laplacian noise scaled to $\frac{1 - p}{1 - 2p}$ before sending the estimator to the data curator. In other words, the data curator receives the noisy version of $f_u(u, w)$ denoted by

$$\tilde{f}_u(u, w) = \sum_{v \in N(u, G)} \frac{\mathcal{A}'[v, w] - p}{1 - 2p} + \text{Lap} \left(\frac{1 - p}{(1 - 2p)\varepsilon_2} \right) \quad (3)$$

LEMMA 1. $\tilde{f}_u(u, w)$ in Equation 3 is an unbiased estimate of $C_2(u, w)$, i.e., $\mathbb{E}(\tilde{f}_u(u, w)) = C_2(u, w)$.

PROOF. Since $\mathbb{E}(\frac{\mathcal{A}'[v, w] - p}{1 - 2p}) = \mathcal{A}[v, w]$, the expected value of the first term in $\tilde{f}_u$ is $C_2(u, w)$. The second term represents the noise drawn from the Laplacian distribution with an expected value of zero. Thus, $\tilde{f}_u(u, w)$ is unbiased. $\square$

The MultiR-SS algorithm. In this part, we present the Multiple-round Single Source algorithm (MultiR-SS) which involves two rounds of interaction between the vertices and the data curator

Algorithm 3: The MultiR-SS algorithm**Input:** G : a bipartite graph; ε : a privacy budget; u, w : two query vertices**Output:** $\tilde{f}_u(u, w)$

```

1  split privacy budget  $\varepsilon$  into  $\varepsilon_1$  and  $\varepsilon_2$  evenly;
   // round 1:
2   $p \leftarrow \frac{1}{1+e^{\varepsilon_1}}$ ;
3  foreach  $j \in$  the opposite layer from  $u$  and  $w$  do
4      perturb  $\mathcal{A}'[u, j] \leftarrow \begin{cases} 1 - \mathcal{A}[u, j], & \text{w.p. } \frac{1}{1+e^{\varepsilon}} \\ \mathcal{A}[u, j] & \text{w.p. } \frac{e^{\varepsilon}}{1+e^{\varepsilon}} \end{cases}$ 
5  send noisy edges to the data curator;
6   $G'_{\varepsilon_1} \leftarrow$  the noisy graph constructed from  $\mathcal{A}'[u, j]$ ;
   // round 2:
7   $S_1 \leftarrow 0$ ;  $S_2 \leftarrow 0$ ;
8  foreach  $v \in N(u, G)$  do
9      if  $(v, w) \in E(G'_{\varepsilon_1})$  then
10          $S_1 \leftarrow S_1 + 1$ ;
11     else
12          $S_2 \leftarrow S_2 + 1$ ;
13   $\tilde{f}_u(u, w) \leftarrow S_1 \times \frac{1-p}{1-2p} - S_2 \times \frac{p}{1-2p}$ ;
14   $\tilde{f}_u(u, w) \leftarrow \tilde{f}_u(u, w) + \text{Lap}\left(\frac{1-p}{(1-2p)\varepsilon_2}\right)$ ;
15 return  $\tilde{f}_u(u, w)$ ;

```

and returns the unbiased estimator $\tilde{f}_u(u, w)$ derived in Lemma 1. The detailed steps are summarized in Algorithm 3. Initially, MultiR-SS splits the privacy budget ε into ε_1 and ε_2 evenly. In the first round, randomized responses are applied to both u and w to generate noisy edges, which are then transmitted to the data curator (Lines 3-6). Then, G'_{ε_1} is constructed from these noisy edges. In the second round, MultiR-SS visits the neighbors of u on G and counts how many are connected to w on G'_{ε_1} . Upon termination of the for-loop (Lines 9-13), S_1 represents the $|N(u, G) \cap N(w, G'_{\varepsilon_1})|$ and S_2 represents the $|N(u, G) \setminus N(w, G'_{\varepsilon_1})|$. Based on S_1 and S_2 , MultiR-SS computes $\tilde{f}_u(u, w)$ and add Laplacian noise scaled to $\frac{1-p}{(1-2p)\varepsilon_2}$ (Lines 14, 15). Compared to OneR that considers all vertices on the opposite layer from u and w on the noisy graph, MultiR-SS limits the search scope for the common neighbors of u and w to the local neighbors of u , which results in a substantial reduction in L2 loss.

EXAMPLE 3. Consider the bipartite graph in Fig. 1. In Fig. 4, we illustrate the construction of the single-source estimators where $u = u_1$ and $w = u_2$. The outlines of u_1 and w_2 are highlighted in red. A privacy budget ε_1 is allocated to randomized responses for u_1 and u_2 . In Fig. 4, each query vertex can download the noisy edges from the other query vertex and integrate them with its neighbors. In the local perspective of u_1 , the solid lines represent edges between u_1 and its neighbors, while the dashed lines represent noisy edges from u_2 . The vertices shaded in grey are candidates for common neighbors between u_1 and u_2 in MultiR-SS, which includes the neighbors of u_1 in the original graph. Note that this is much smaller than the candidate pool identified by OneR, which includes all vertices on the opposite layer from the query vertices. Based on the formula for $\tilde{f}_u$, it only relies on three Bernoulli variables: $\mathcal{A}'[v_1, u_2]$, $\mathcal{A}'[v_2, u_2]$, and $\mathcal{A}'[v_4, u_2]$. Similarly, we can derive $\tilde{f}_w$ based on four random variables: $\mathcal{A}'[v_1, u_2]$, $\mathcal{A}'[v_2, u_2]$, $\mathcal{A}'[v_4, u_2]$, and $\mathcal{A}'[v_{100}, u_2]$. The reliance on fewer random variables accounts for the smaller expected L2 loss of MultiR-SS compared to OneR.

Fig. 5. Illustration of the L2 loss of f^* when $\varepsilon = 2$.

Theoretical analysis for MultiR-SS. Without loss of generality, we assume that the query vertices u and w are in $L(G)$. First, we analyze the computational time complexity of Algorithm 3 (MultiR-SS). On the vertex side, the time costs incurred by randomized responses is $O(n_1)$, where $n_1 = |U(G)|$. On the curator side, visiting the neighbors of u in G to compute $\tilde{f}_u(u, w)$ takes $O(\deg(u, G))$ time. The overall time complexity is $O(n_1)$. Then, we analyze the communications costs of MultiR-SS, which include (1) sending the noisy edges from w and downloading them to vertex u and (2) sending the single-source estimator $\tilde{f}_u(u, w)$ to the data curator. The dominating cost is incurred by the step (1). Note that the expected number of noisy edges from vertex w is $d_w \times (1-p) + (n_1 - d_w) \times p$, where $p = \frac{1}{1+e^{\varepsilon_1}}$. Thus, the overall communication cost is $O(\frac{e^{\varepsilon_1}-1}{e^{\varepsilon_1}+1}d_w + \frac{n_1}{1+e^{\varepsilon_1}})$. In the following, we analyze the expected L2 loss of MultiR-SS.

THEOREM 6. *The expected L2 loss of $\tilde{f}_u(u, w)$ in Equation 3 is $O(\frac{e^{\varepsilon_1}}{(1-e^{\varepsilon_1})^2}(d_u + \frac{2e^{\varepsilon_1}}{\varepsilon_2^2}))$. Here d_u represents the degree of u in G .*

PROOF. Since $\tilde{f}_u(u, w)$ is an unbiased estimator, its L2 loss equals its variance. The variance of $\tilde{f}_u(u, w)$ consists of two parts: $f_u(u, w)$ and the Laplacian noise. First, it immediately follows that the variance from $f_u(u, w)$ is $\frac{p(1-p)}{(1-2p)^2}d_u$ based on Equation 1. This is because each $\phi(u, w)$ is independent of each other and there is no covariance involved. In addition, the variance from the Laplacian noise is:

$$\text{Var}(\text{Lap}(\frac{\Delta(\tilde{f}_u(u, w))}{\varepsilon_2})) = 2(\frac{1-p}{(1-2p)\varepsilon_2})^2 = \frac{2(1-p)^2}{(1-2p)^2\varepsilon_2^2}$$

Since $f_u(u, w)$ and the Laplacian noise are independent, the expected L2 loss of $\tilde{f}_u$ is $\frac{p(1-p)}{(1-2p)^2}d_u + \frac{2(1-p)^2}{(1-2p)^2\varepsilon_2^2}$. Substituting $p = \frac{1}{1+e^{\varepsilon_1}}$ into the above expression completes the proof. $\square$

Since the L2 loss of $\tilde{f}_u(u, w)$ is no longer dependent on n_1 , the data utility of MultiR-SS is significantly improved compared to OneR. In addition, we check whether Algorithm 3 satisfies the privacy requirements of ε -edge LDP in the following theorem.

THEOREM 7. *Given a bipartite graph G and a privacy budget ε , Algorithm 3 satisfies ε -edge LDP.*

PROOF. We use the *Sequential Composition* theorem [21] to prove that Algorithm 3 satisfies ε -edge LDP. In the first round, generating the noisy edges via randomized responses satisfies ε_1 -edge LDP (Lines 2-6). In the second round, Lines 7-13 are conducted locally by the vertex u . Then, the Laplace mechanism (Line 14) is applied w.r.t. a privacy budget of ε_2 to construct the unbiased estimator $\tilde{f}_u$, which satisfies ε_2 -edge LDP. By the sequential composition property of edge LDP, Algorithm 3 satisfies ε -edge LDP ($\varepsilon = \varepsilon_1 + \varepsilon_2$). $\square$

4.2 A double-source estimator for $C_2(u, w)$

The single-source estimator $\tilde{f}_u(u, w)$ only involves the neighborhood of u . Similarly, we can develop another unbiased estimator, $\tilde{f}_w(u, w) = \sum_{v \in N(w, G)} \phi(v, u)$, by applying the same process to the neighborhood of w . This raises a natural question: How can we integrate these estimators to further minimize L2 loss while maintaining unbiasedness? Examining the loss of L2 of $\tilde{f}_u(u, w)$ in Theorem 6, we can see that it consists of the first term representing the error incurred by randomized responses, and the second term representing the error incurred by Laplacian noise. On the one hand, if we only minimize the first term, we could always choose the estimator between $\tilde{f}_u$ and $\tilde{f}_w$

whose corresponding query vertex has a smaller degree. On the other hand, if we only focus on minimizing the second term, we could take an average of $\tilde{f}_u$ and $\tilde{f}_w$ and the Laplacian noise of the resulting estimator will be reduced by half. To balance both objectives, we propose a double-source estimator f^* by taking a weighted average of $\tilde{f}_u$ and $\tilde{f}_w$, i.e., $f^* = \alpha\tilde{f}_u + (1 - \alpha)\tilde{f}_w$ ($\alpha \in [0, 1]$). Here α is the weighting parameter that adjusts the contribution of $\tilde{f}_u$ and $\tilde{f}_w$. By analyzing the L2 loss of f^* , we introduce the Multiple-round Double Source algorithm (MultiR-DS), which enhances data utility by optimizing the allocation of privacy budget and balancing the contribution of $\tilde{f}_u$ and $\tilde{f}_w$.

Properties of the double-source estimator f^* . Given that f^* is a weighted average of $\tilde{f}_u$ and $\tilde{f}_w$, its unbiasedness directly stems from the principle of linearity in expected values, i.e., $\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$. Thus, based on the bias-variance decomposition, its L2 loss equals its variance, as analyzed in the following theorem.

THEOREM 8. *The L2 loss of $f^* = \alpha\tilde{f}_u + (1 - \alpha)\tilde{f}_w$ ($\alpha \in [0, 1]$) is*

$$\frac{e^{\epsilon_1}}{(1 - e^{\epsilon_1})^2} \left((\alpha^2 d_u + (1 - \alpha)^2 d_w) + \frac{2(\alpha^2 + (1 - \alpha)^2)e^{\epsilon_1}}{\epsilon_2^2} \right)$$

Here d_u and d_w represent the degrees of u and w in G .

PROOF.

$$\begin{aligned} l2(f^*, C_2(u, w)) &= \text{Var}(f^*) = \alpha^2 \text{Var}(\tilde{f}_u) + (1 - \alpha)^2 \text{Var}(\tilde{f}_w) \\ &= \frac{p(1 - p)}{(1 - 2p)^2} (\alpha^2 d_u + (1 - \alpha)^2 d_w) + \frac{2(1 - p)^2}{(1 - 2p)^2 \epsilon_2^2} (\alpha^2 + (1 - \alpha)^2) \end{aligned}$$

Note that $\tilde{f}_u$ depends on the noisy edges connected to w , while $\tilde{f}_w$ depends on the noisy edges connected to u . Since they depend on disjoint edges in the noisy graph, $\tilde{f}_u$ and $\tilde{f}_w$ are independent, and their covariance $\text{Cov}(\tilde{f}_u, \tilde{f}_w) = 0$. Thus, the first step holds. Also, the expected L2 loss of $\tilde{f}_w$ is the same as $\tilde{f}_u$ where d_u is replaced by d_w . Substituting $p = \frac{1}{1 + e^{\epsilon_1}}$ completes the proof. $\square$

Based on Theorem 8, the L2 loss of f^* is a function of ϵ_1 and α , where d_u and d_w are constants. We denote it by $l2(f^*, C_2(u, w)) := F(\epsilon_1, \alpha)$. To obtain estimates for d_u and d_w , we can apply the Laplace mechanism in an additional round using a small privacy budget (ϵ_0). If the estimates of d_u and d_w are negative, we can estimate the average vertex degree in $L(G)$ and substitute them. To minimize this loss, we seek values of $\epsilon_1 \in (0, \epsilon)$ and $\alpha \in [0, 1]$ that minimize $F(\epsilon_1, \alpha)$. We discover that F reaches its global minimum if and only if its partial derivatives $\frac{\partial F}{\partial \alpha} = \frac{\partial F}{\partial \epsilon_1} = 0$. However, this results in a transcendental equation that lacks analytical solutions. Thus, we resort to Newton's method [12] for high-precision approximate solutions. By optimizing ϵ_1 and α , the resulting L2 loss of f^* will be lower than that of both single-source estimators $\tilde{f}_u$ and $\tilde{f}_w$.

We could also optimize the expected L2 loss for MultiR-SS, which is a function of ϵ_1 and $\text{deg}(u)$. Specifically, we could spend a small privacy budget (ϵ_0) to estimate $\text{deg}(u)$ and then apply Newton's method to find the best privacy budgets (ϵ_1 and ϵ_2) that minimize the expected L2 loss of MultiR-SS ($\epsilon = \sum_{i=0}^2 \epsilon_i$). In practice, this implementation only outperforms the current MultiR-SS with an even separation of privacy budget ($\epsilon_1 = \epsilon_2$) when the degree of u is large. In addition, it is a special case of MultiR-DS where $\alpha = 1$.

The MultiR-DS algorithm. In this part, we present the Multiple-round Double Source algorithm (MultiR-DS) which uses an additional round compared to MultiR-SS to estimate d_u and d_w and estimate the L2 loss of f^* . The detailed steps are outlined in Algorithm 4. In the first round, MultiR-DS uses a small privacy budget ϵ_0 and applies the Laplace mechanism to obtain unbiased estimates of d_u and d_w (Lines 1-3). Here the global sensitivity of d_u (d_w) is one because adding

Algorithm 4: The MultiR-DS algorithm**Input:** G : a bipartite graph; ε : a privacy budget; u, w : two query vertices**Output:** $\tilde{f}_u(u, w)$

// round 1:

```

1   $\varepsilon_0 \leftarrow \varepsilon \times 0.05$ ;
2   $d_u \leftarrow \deg(u, G) + \text{Lap}(\frac{1}{\varepsilon_0})$ ;
3   $d_w \leftarrow \deg(w, G) + \text{Lap}(\frac{1}{\varepsilon_0})$ ;
4   $d' \leftarrow$  the average vertex degree on the same layer as  $u$ ;
5  correct  $d_u$  and  $d_w$  with  $d'$ ;
6  find  $\alpha$  and  $\varepsilon_1$  that minimizes  $\text{Var}(f^*)$ ;
  // round 2:
7   $p \leftarrow \frac{1}{1+e^{\varepsilon_1}}$ ;
8  foreach  $i \in \{u, w\}$  do
9    foreach  $j \in$  the opposite layer from  $u$  and  $w$  do
10      $\text{perturb } \mathcal{A}'[i, j] \leftarrow \begin{cases} 1 - \mathcal{A}[i, j], & \text{w.p. } \frac{1}{1+e^{\varepsilon}} \\ \mathcal{A}[i, j] & \text{w.p. } \frac{e^{\varepsilon}}{1+e^{\varepsilon}} \end{cases}$ 
11 send noisy edges to the data curator;
12  $G'_{\varepsilon_1} \leftarrow$  the noisy graph constructed from  $\mathcal{A}'[i, j]$ ;
  // round 3:
13  $\varepsilon_2 \leftarrow \varepsilon - \varepsilon_0 - \varepsilon_1$ ;
14  $\tilde{f}_u(u, w) \leftarrow$  the estimator by running Lines 8-15 of Algorithm 3;
15  $\tilde{f}_w(u, w) \leftarrow$  the estimator by running Lines 8-15 of Algorithm 3 with  $u$  and  $w$  switched;
16 return  $\alpha \tilde{f}_u + (1 - \alpha) \tilde{f}_w$ ;
```

or deleting an edge from the neighbor list of u (w) changes d_u (d_w) by at most one. Due to the Laplacian noise, the reported d_u and d_w could be negative. In this case, we correct for any negative value with the estimated average degree of the vertices on the same side as u and w (Lines 4, 5). Then, the MultiR-DS algorithm invokes Newton's method to find the pair of α and ε_1 that minimizes the estimated L2 loss of f^* . In the second round, randomized responses are applied to u and w with respect to ε_1 , leading to the noisy graph G'_{ε_1} (Lines 7-12). In the third round, MultiR-DS allocates the remaining privacy budget ε_2 to construct unbiased estimators $\tilde{f}_u$ and $\tilde{f}_w$ from the local neighborhoods of u and w . Specifically, $\tilde{f}_u$ is derived by executing Lines 8-15 of MultiR-SS, while $\tilde{f}_w$ is computed similarly by visiting the neighbors of w in G (Lines 14-15). Note that when constructing $\tilde{f}_u$ and $\tilde{f}_w$, the global sensitivity analysis is the same as in MultiR-SS. In other words, the global sensitivity of $\frac{1-p}{1-2p}$ for each single-source estimator is applied to both $\tilde{f}_u$ and $\tilde{f}_w$ upon construction. In the end, MultiR-DS returns the weighted average of $\tilde{f}_u$ and $\tilde{f}_w$ where the parameter α is computed in the first round (Line 16).

Theoretical analysis for MultiR-DS. Without loss of generality, we assume that u and $w \in L(G)$. First, we analyze the computational time complexity of MultiR-DS. On the vertex side, estimating the average degree of the vertices in $L(G)$ takes $O(n_2)$ time. When constructing the noisy graph, the time costs incurred by the randomized responses are $O(n_1)$. On the curator side, visiting the neighbors of u and w to compute $\tilde{f}_u$ and $\tilde{f}_w$ takes $O(\deg(u, G) + \deg(w, G))$ time. Thus, the overall time complexity is $O(n)$.

We then analyze the communication costs of MultiR-DS, which include: (1) sending the noisy degree of all vertices in $L(G)$, (2) sending the noisy edges from w and downloading them to vertex

u , (3) sending the noisy edges from u and downloading them to vertex w , and (4) sending two single-source estimators $\tilde{f}_u$ and $\tilde{f}_w$ to the data curator. Step (1) incurs communication costs of $O(n_2)$. The communication costs for Step (2) and Step (3) are proportional to the expected number of noisy edges from u and w , which is $(d_u + d_w) \times (1 - p) + 2(n_1 - d_w) \times p$, where $p = \frac{1}{1+e^{\epsilon_1}}$. Step (4) incurs a communication cost of $O(1)$. Thus, the overall communication cost is $O(n_2 + \frac{e^{\epsilon_1}-1}{e^{\epsilon_1}+1}(d_w + d_u) + \frac{2n_1}{1+e^{\epsilon_1}})$.

Since the expected L2 loss of MultiR-DS has been analyzed in Theorem 8, we compare it with the expected L2 loss of MultiR-SS in the following theorem.

THEOREM 9. *The minimum L2 loss incurred by the double-source estimator $f^* = \alpha \tilde{f}_u + (1 - \alpha) \tilde{f}_w$ is less than or equal to the L2 loss incurred by both single-source estimators $\tilde{f}_u$ and $\tilde{f}_w$.*

$$\min_{\epsilon_1, \alpha} l_2(f^*, C_2(u, w)) \leq \min(l_2(\tilde{f}_u, C_2(u, w)), l_2(\tilde{f}_w, C_2(u, w)))$$

PROOF. Let L^* be the minimized expected L2 loss of the double-source estimator f^* . To prove the above inequality, we need to prove that $L^* \leq l_2(\tilde{f}_u, C_2(u, w))$ and $L^* \leq l_2(\tilde{f}_w, C_2(u, w))$. By construction, $\tilde{f}_u$ is a special case of f^* where $\alpha = 1$, i.e., $\tilde{f}_u = f^*|_{\alpha=1}$. Hence, for any privacy budget allocations (ϵ_1 and ϵ_2), we have $L^* \leq l_2(\tilde{f}_u, C_2(u, w))$. Similarly, $\tilde{f}_w$ is also a special case of f^* where $\alpha = 0$, i.e., $\tilde{f}_w = f^*|_{\alpha=0}$. We also obtain $L^* \leq l_2(\tilde{f}_w, C_2(u, w))$. Combining these two inequalities completes the proof. $\square$

To illustrate the comparison, we plot the L2 loss of f^* , $\tilde{f}_u$, and $\tilde{f}_w$ against varying ϵ_1 values in Fig. 5, where $\epsilon = 2$. The blue curve labeled “ $\alpha = 0$ ” represents the L2 loss of $\tilde{f}_w$. The red curve labeled “ $\alpha = 1$ ” represents the L2 loss of $\tilde{f}_u$. The green curve labeled “ $\alpha = 0.5$ ” represents the unbiased estimator $f' = (\tilde{f}_u + \tilde{f}_w)/2$. The grey horizontal line marks the global minimum L2 loss for f^* . On the left, when $d_u = 5$ and $d_w = 10$, f' outperforms $\tilde{f}_u$ and $\tilde{f}_w$ and reaches the global minimum. On the right, when d_u and d_w are more imbalanced, $\tilde{f}_u$ becomes the best estimator, reaching the global minimum. None of the single-source estimators or their average can consistently reach the minimized L2 loss of f^* for all query vertex pairs. This is due to the flexibility of f^* in adjusting the privacy budget allocation and the weighting of $\tilde{f}_u$ and $\tilde{f}_w$.

In the following theorem, we verify the compliance of Algorithm 4 to ϵ -edge LDP.

THEOREM 10. *Given a bipartite graph G and a privacy budget ϵ , Algorithm 4 satisfies ϵ -edge LDP.*

PROOF. We use the *Sequential Composition* and *Parallel Composition* theorems for ϵ -edge LDP [21]. Parallel composition theorem states that if different ϵ -edge LDP algorithms are applied to disjoint datasets with privacy budgets ϵ_i , the composite algorithm satisfies $\max_i \epsilon_i$ -LDP. In the first round, each vertex reports its degree using the Laplace mechanism, achieving ϵ_0 -edge LDP. By parallel composition, this round satisfies ϵ_0 -edge LDP. In the second round, randomized responses provide ϵ_1 -edge LDP. In the third round, building $\tilde{f}_u$ and $\tilde{f}_w$ satisfies ϵ_2 -edge LDP. By parallel composition, this round satisfies ϵ_2 -edge LDP. By sequential composition, Algorithm 4 satisfies ϵ -LDP with $\epsilon = \epsilon_0 + \epsilon_1 + \epsilon_2$. $\square$

Summary of the expected L2 losses of all algorithms. In Table 3, we summarize the expected L2 losses of all privacy-preserving algorithms for estimating the number of common neighbors. The expected L2 loss of OneR is smaller than that of Naive, with OneR having an expected L2 loss of $O(n_1)$ compared to Naive’s $O(n_1^2)$. In addition, the expected L2 losses of MultiR-SS and MultiR-DS are lower than those of Naive and OneR because they do not depend on the number of vertices in the graph. Between MultiR-DS and MultiR-SS, as analyzed in Theorem 9, the minimized loss for MultiR-DS is smaller than MultiR-SS because MultiR-SS is a special case of MultiR-DS where

$\alpha = 0$ or $\alpha = 1$. Note that for OneR, MultiR-SS, and MultiR-DS, which are unbiased estimators, their expected L2 losses can offer insight into their deviation from the true value by applying *Chebyshev's inequality* [38]. For instance, for the OneR algorithm, we know $\mathbb{E}(\tilde{f}_2(u, w)) = C_2(u, w)$ and that $\text{Var}(\tilde{f}_2(u, w)) = \frac{p^2(1-p)^2}{(1-2p)^4}n_1 + \frac{p(1-p)}{(1-2p)^2}(d_w + d_u)$. Chebyshev's inequality states that for any $k > 0$:

$$P\left(|\tilde{f}_2(u, w) - C_2(u, w)| \geq k\sqrt{\text{Var}(\tilde{f}_2(u, w))}\right) \leq \frac{1}{k^2}.$$

Similar probabilistic bounds can be derived for the MultiR-SS and MultiR-DS algorithms based on their expected L2 losses.

Table 2. Summary of Datasets

Dataset	Upper	Lower	E	U	L
Rmwiki (RM)	User	Article	58.0K	1.2K	8.1K
Collaboration (AC)	Author	Paper	58.6K	16.7K	22.0K
Occupation (OC)	Person	Occupation	250.9K	127.6K	101.7K
Bag-kos (DA)	Document	Word	353.2K	3.4K	6.9K
Bpywiki (BP)	User	Article	399.7K	1.3K	57.9K
Tewiktionary (MT)	User	Article	529.6K	495	121.5K
Bookcrossing (BX)	User	Book	1.1M	105.3K	340.5K
Stackoverflow (SO)	User	Post	1.3M	545.2K	96.7K
Team (TM)	Athlete	Team	1.4M	901.2K	34.5K
Wiki-en-cat (WC)	Article	Category	3.8M	1.9M	182.9K
Movielens (ML)	User	Movie	10.0M	69.9K	10.7K
Epinions (ER)	User	Product	13.7M	120.5K	755.8K
Netflix (NX)	User	Movie	100.5M	480.2K	17.8K
Delicious-ui (DUI)	User	Url	101.8M	833.1K	33.8M
Orkut (OG)	User	Group	327.0M	2.8M	8.7M

Table 3. Summary of time costs, expected L2 losses, and communication costs

Algorithm	Time cost	Unbiased	Expected L2 loss	Communication cost
Naive	$O(n_1)$	$\times$	$O(\frac{n_1^2 e^{\epsilon}}{(1+e^{\epsilon})^4})$	$O(\frac{e^{\epsilon}-1}{e^{\epsilon}+1}(d_u + d_w) + \frac{2n_1}{1+e^{\epsilon}})$
OneR	$O(n_1)$	$\checkmark$	$O(\frac{n_1 e^{2\epsilon}}{(1-e^{\epsilon})^4})$	$O(\frac{e^{\epsilon}-1}{e^{\epsilon}+1}(d_u + d_w) + \frac{2n_1}{1+e^{\epsilon}})$
MultiR-SS	$O(n_1)$	$\checkmark$	$O(\frac{e^{\epsilon_1}}{(1-e^{\epsilon_1})^2}(d_u + \frac{2e^{\epsilon_1}}{\epsilon_2^2}))$	$O(\frac{e^{\epsilon_1}-1}{e^{\epsilon_1}+1}d_w + \frac{n_1}{1+e^{\epsilon_1}})$
MultiR-DS	$O(n)$	$\checkmark$	$O(\frac{e^{\epsilon_1}}{(1-e^{\epsilon_1})^2}(\alpha^2 d_u + (1-\alpha)^2 d_w) + \frac{2e^{2\epsilon_1}}{(1-e^{\epsilon_1})^2} \frac{\alpha^2 + (1-\alpha)^2}{\epsilon_2^2})$	$O(n_2 + \frac{e^{\epsilon_1}-1}{e^{\epsilon_1}+1}(d_w + d_u) + \frac{2n_1}{1+e^{\epsilon_1}})$

* Without loss of generality, we assume $u, w \in L(G)$. ϵ is the overall privacy budget. In MultiR-SS and MultiR-DS, ϵ_1 represents the privacy budget for randomized responses, while ϵ_2 denotes the budget for the Laplace mechanism. $\alpha \in [0, 1]$ adjusts the contribution of $\tilde{f}_u$ and $\tilde{f}_w$. d_u and d_w represent the degrees of u and w in G . $n_1 = |U(G)|$, $n_2 = |L(G)|$, and $n = |U(G) \cup L(G)|$.

5 Experimental evaluation

In this section, we evaluate the proposed common neighbor estimation algorithms under ϵ -edge LDP through experiments.

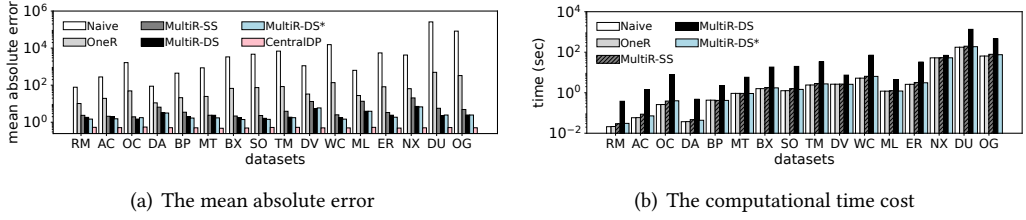
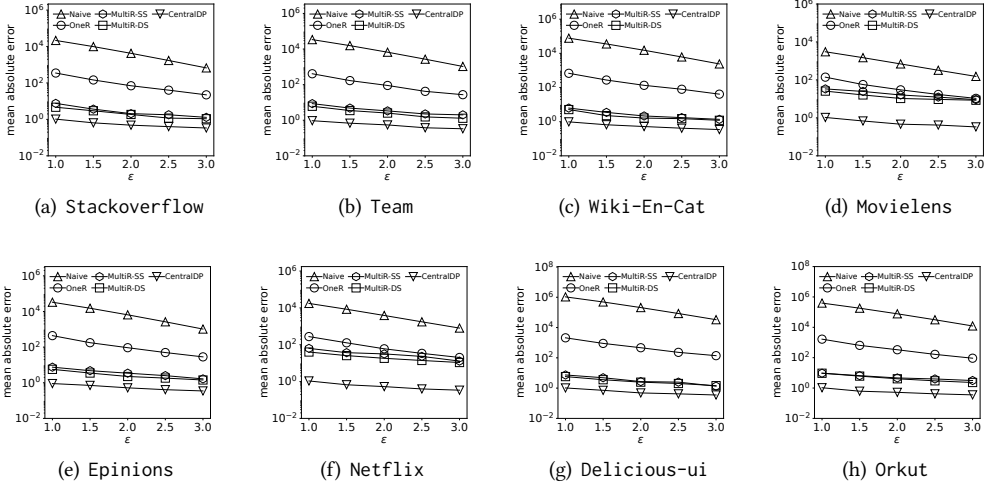
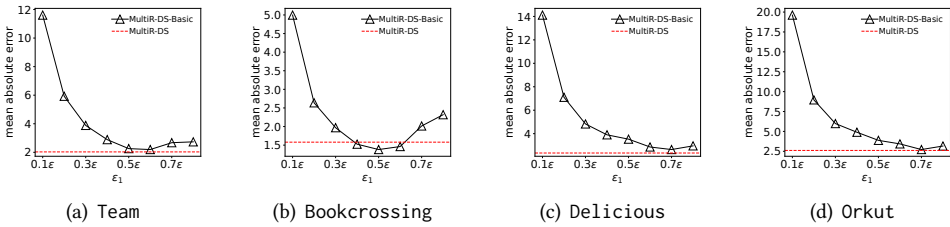
Fig. 6. Performance on different datasets ($\epsilon = 2$)Fig. 7. Effect of ϵ on the mean relative errors of Naive, OneR, and MultiR-DS.

Fig. 8. Effectiveness of privacy budget allocation optimization.

5.1 Experimental Settings

Datasets. We use 15 datasets from KONECT (<http://konect.cc/>). Table 2 shows the statistics of the datasets. $|U|$ and $|L|$ are the number of vertices in the upper and lower layers. $|E|$ is the number of edges in the graph.

Algorithms. We evaluate the following common neighborhood estimation algorithms under ϵ -edge LDP.

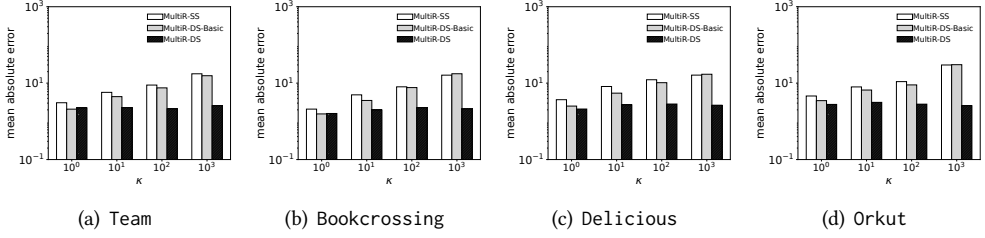


Fig. 9. Effectiveness of MultiR-DS.

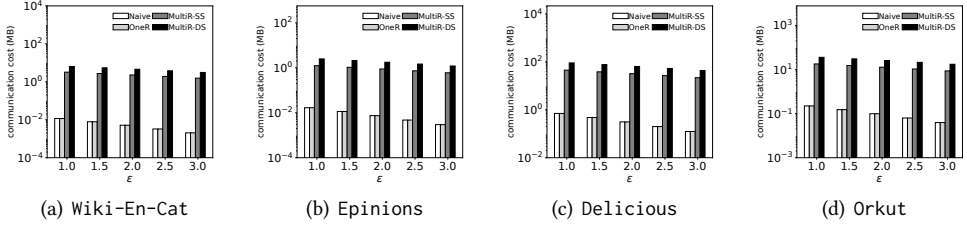


Fig. 10. Communication costs.

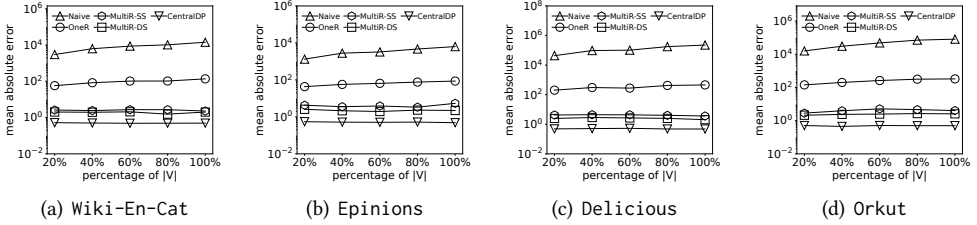


Fig. 11. Effect of the number of vertices.

- Naive: the algorithm that returns the number of common neighbors between u and w on the noisy graph G' ;
- OneR: the one-round algorithm returns an unbiased estimate of $C_2(u, w)$ based on the noisy graph G' ;
- MultiR-SS: the multiple-round single-source algorithm that returns the unbiased estimator $\tilde{f}_u(u, w)$ by utilizing the local neighborhood of u ;
- MultiR-DS: the multiple-round double-source algorithm that returns the unbiased estimator $\alpha\tilde{f}_u + (1 - \alpha)\tilde{f}_w$ ($\alpha \in [0, 1]$) by utilizing the local neighborhoods of both u and w .

We also implement two variants of MultiR-DS: MultiR-DS-Basic and MultiR-DS*. MultiR-DS-Basic returns the average of the two single-source estimators $\tilde{f}_u$ and $\tilde{f}_w$ ($\frac{\tilde{f}_u + \tilde{f}_w}{2}$). It spends ϵ_1 on noisy graph construction and $1 - \epsilon_1$ on the Laplace mechanism and does not estimate $\deg(u)$ or $\deg(w)$. Similarly to MultiR-DS, MultiR-DS* returns $\alpha\tilde{f}_u + (1 - \alpha)\tilde{f}_w$ ($\alpha \in [0, 1]$) and adopts the same optimizations for the allocation of privacy budgets to find ϵ_1 and α . The difference is that MultiR-DS* assumes that the vertex degrees are public and it does not need an additional round for vertex degree estimation.

To better evaluate the edge LDP algorithms, we also implement the CentralDP algorithm under the centralized model, which assumes the data curator has access to the entire bipartite graph. CentralDP directly applies the Laplacian mechanism to the number of common neighbors of two query vertices. Since the global sensitivity of $C_2(u, w)$ in the central model is 1, CentralDP returns $C_2(u, w) + \text{Lap}(\frac{1}{\epsilon})$. All algorithms are implemented in C++. The experiments are run on a Linux server with an Intel Xeon 6342 processor and 512GB memory.

Parameter settings. By default, the privacy budget ϵ is set to 2. We also allow it to vary from 1 to 3 with increments of 0.5. For MultiR-SS and MultiR-DS-Basic, ϵ_1 is set to 0.5ϵ by default. For MultiR-DS, we set $\epsilon_0 = 0.05\epsilon$ for degree estimations. For each algorithm, we uniformly sample 100 vertex pairs on the same layer and report the mean absolute error, the average of the absolute differences between the predicted and true values across all sampled vertex pairs. To evaluate the performance of MultiR-DS, we use κ to quantify the imbalance between two vertex degrees. Specifically, on a given pair of vertices (u and w) with the parameter κ , we have $\max(\deg(u), \deg(w)) > \kappa \times \min(\deg(u), \deg(w))$.

Evaluate the effectiveness of edge LDP algorithms across different datasets. In Fig. 6(a), we report the performances of the edge LDP algorithms including Naive, OneR, MultiR-SS, MultiR-DS, and MultiR-DS* on 100 uniformly sampled vertex pairs when $\epsilon = 2$. Note that we also include the performance of CentralDP under the centralized model for comparison. First, we observe that the multiple-round algorithms (MultiR-SS, MultiR-DS, and MultiR-DS*) significantly outperform Naive and OneR across all datasets. Specifically, MultiR-SS and MultiR-DS achieve mean absolute errors lower by up to four and two orders of magnitude, respectively. This is because MultiR-SS and MultiR-DS address the overcounting issue due to the dense noisy graph with Naive by deriving unbiased estimates. Meanwhile, compared to the OneR algorithm that produces unbiased estimates by considering all vertices on the opposite layers as possible common neighbors, MultiR-SS and MultiR-DS induce much smaller mean absolute errors by reducing the candidate pool common neighbors to the neighbors of query vertices. We also observe that MultiR-DS consistently produces smaller mean absolute errors than MultiR-SS. For example, on Netflix, the mean absolute error of MultiR-DS is approximately one-fifth that of MultiR-SS. This is because MultiR-DS integrates the two single-source estimators and dynamically adjusts the privacy budget allocation based on the query vertices. In Fig. 6(a), we observe that MultiR-DS* generally produces slightly smaller mean absolute errors compared to MultiR-DS. This is because MultiR-DS* does not need to spend an additional privacy budget for degree estimation, which leads to more privacy budgets for noisy graph construction and the Laplace mechanism (i.e., ϵ_1 and ϵ_2 becomes larger). We also observe that OneR achieves much lower mean absolute errors than Naive because OneR leverages flipping probability to obtain unbiased estimators, which mitigates the over-counting issue in Naive.

In addition, CentralDP results in lower errors than all algorithms with edge LDP. This illustrates the limitations of the local model in terms of data utility due to stronger privacy guarantees.

Evaluate the computational time costs across datasets. In Fig. 6(b), we report the computational time costs of Naive, OneR, MultiR-SS, MultiR-DS, and MultiR-DS* on 100 vertex pairs when $\epsilon = 2$. Note that our evaluation focuses on the computational time costs incurred by both the vertex side and the data curator side. We can observe that the time costs of Naive, OneR, and MultiR-SS remain relatively comparable while MultiR-DS requires more time. This is because the time complexities of Naive, OneR, and MultiR-SS depend on the number of vertices on the opposite layers of the query vertices i.e., $O(n_1)$, which is incurred by noisy graph construction. Since MultiR-DS needs an additional $O(n_2)$ time for the estimation of the average degree, its total time complexity becomes $O(n)$ and exceeds the others. Despite this, MultiR-DS remains highly efficient and can scale effectively to bipartite graphs with 300 million edges (i.e., Orkut). Also, in practice, the time required for average degree estimation is distributed across vertices. Additionally,

we observe that the MultiR-DS* algorithm runs faster than MultiR-DS and incurs comparable time costs to MultiR-SS. This is because MultiR-DS* does not need an additional round to estimate the vertex degrees.

Evaluate the effect of the privacy budget ϵ . As shown in Fig. 7, we report the mean absolute errors of Naive, OneR, and MultiR-DS on 8 datasets, as ϵ varies. Note that we also include CentralDP under central differential privacy for comparison. We observe that all algorithms produce smaller mean absolute errors as ϵ increases, which is consistent with our L2 loss analysis. As ϵ increases, the difference between any noisy graph constructed by randomized responses and the input graph becomes smaller. Another pattern is that the multiple-round algorithms (MultiR-SS and MultiR-DS) significantly outperform Naive and OneR, with mean absolute errors up to four orders of magnitude lower. This is because the expected L2 losses of Naive and OneR are $O(n_1^2)$ and $O(n_1)$, respectively, while the expected L2 losses of MultiR-SS and MultiR-DS only depend on vertex degrees. MultiR-DS consistently outperforms MultiR-SS on varying values of ϵ because MultiR-DS integrates both single-source estimators and employs privacy budget allocation optimizations for minimized L2 loss. We also observe that OneR consistently outperforms Naive as the privacy budget increases. As expected, CentralDP produces smaller mean absolute errors than algorithms under edge LDP, which has stronger privacy guarantees.

Evaluate the effect of privacy budget allocation optimization on MultiR-DS. In Fig. 8, we present the mean absolute errors of MultiR-DS-Basic in four datasets, as ϵ_1 ranges from 0.1ϵ to 0.7ϵ , where $\epsilon = 2$ and $\epsilon_2 = \epsilon - \epsilon_1$. Note that MultiR-DS-Basic does not employ the privacy budget allocation optimization. In contrast, MultiR-DS adjusts ϵ_1 and the contribution of two single-source estimators (measured by α) based on the query vertices. We use red dashed horizontal lines to indicate the mean absolute errors associated with MultiR-DS. First, the optimal budget allocation plan varies across datasets and it is unrealistic to fix ϵ_1 and ϵ_2 for all datasets. This is because optimal budget allocation depends on the degrees of the query vertices, as shown in Table 3. Also, for each dataset, the mean absolute error with MultiR-DS is close to or even smaller than the smallest mean absolute error of MultiR-DS-Basic on varying values of ϵ_1 . This implies that MultiR-DS can find ϵ_1 and α that result in near-optimal L2 loss.

Evaluate the effectiveness of MultiR-DS on vertex pairs with imbalanced degrees. In Fig. 9, we report the mean absolute errors of MultiR-SS, MultiR-DS-Basic, and MultiR-DS across four datasets, as κ ranges from 10^0 to 10^3 , with $\epsilon = 2$. Here, κ quantifies the imbalance between two vertex degrees. For MultiR-DS-Basic, ϵ_1 is set to 0.5ϵ . We observe that the mean absolute errors of MultiR-SS and MultiR-DS-Basic increase as κ increases, while the performance of MultiR-DS remains relatively unchanged. This is because MultiR-SS only relies on one query vertex to construct the unbiased estimator $\tilde{f}_u$. Thus, if $\deg(u, G)$ is large, the error increases accordingly, as indicated in Table 3. For MultiR-DS-Basic, it performs slightly better than MultiR-SS when κ is small because it allows $\tilde{f}_u$ and $\tilde{f}_w$ to contribute equally (i.e., $(\tilde{f}_u + \tilde{f}_w)/2$). However, when the vertex degrees are highly imbalanced (κ becomes large), the errors of MultiR-DS-Basic escalate rapidly. Also, neither MultiR-SS nor MultiR-DS-Basic can adjust privacy budget allocations based on the query vertices. In contrast, MultiR-DS uses α to model the contribution of two query vertices and dynamically adjust privacy budgets to minimize L2 loss. (1) If the vertex degrees are large, MultiR-DS allocates more privacy budget to ϵ_1 . (2) If the vertex degrees are imbalanced, MultiR-DS adjusts α so that the query vertex with a smaller degree has a greater contribution.

Evaluate the communication costs of all algorithms. In Fig. 10, we report the communication costs (in MB) of each algorithm averaged across 100 randomly sampled vertex pairs in four datasets, as ϵ varies. We observe that Naive and OneR require approximately the same message sizes. This is because Naive and OneR rely solely on randomized responses to satisfy edge LDP. Given a

fixed ϵ , they apply randomized responses with the same flipping probability, resulting in the same expected number of noisy edges. Also, MultiR-SS and MultiR-DS incur higher communication costs than Naive and OneR, which are incurred by (1) uploading the noisy edges to the data curator (2) downloading the noisy edges to the query vertices (3) sending the estimators ($\tilde{f}_u$ or $\tilde{f}_w$) from the query vertices. For MultiR-DS, the communication costs are higher as it utilizes the noisy edges from both query vertices and also needs to send vertex degree estimated to the data curator. Note that the highest average communication cost for MultiR-DS across datasets is approximately 100 MB, which is modest in practical terms.

Evaluate the effect of the number of vertices. In Fig. 11, we report the mean absolute errors of CentralDP, Naive, OneR, MultiR-SS and MultiR-DS in four datasets as the number of vertices increases. Specifically, on each dataset, we uniformly sample 20%, 40%, 60%, 80%, and 100% of all vertices and run the algorithms on the subgraphs induced by the sampled vertices. The privacy budget ϵ is fixed at 2. First, we observe that the performances of CentralDP, MultiR-SS, and MultiR-DS remain relatively unchanged. This aligns with our analysis for MultiR-SS and MultiR-DS, where their L2 losses depend solely on the allocation of the privacy budget, the degrees of the query vertices, and α , the weighting parameter adjusting the contribution of $\tilde{f}_u$ and $\tilde{f}_w$. For CentralDP, its errors come only from the added Laplacian noise, which is also not related to the number of vertices in the bipartite graph. We also observe that the mean absolute errors of Naive and OneR increase steadily as the number of vertices increases. For instance, on Dui and OG, the mean absolute errors of Naive increase approximately fivefold when the number of vertices increases from $20\% \times |V|$ to $100\% \times |V|$. Meanwhile, the mean absolute errors of OneR show less sensitivity, increasing approximately 2.3 times. This is because the expected L2 losses of Naive and OneR are $O(n_1^2)$ and $O(n_1)$, respectively.

6 Related Work

Here we review the related works on graph analysis under differential privacy.

Graph analysis under differential privacy. Differential privacy is widely adopted for privacy-preserving graph analysis, including releasing degree distributions [5, 14, 15, 32], common neighbor count distribution [30], k -star counting [23, 33], triangle counting [7, 18–20, 31], and core decomposition [6], and graph learning [26, 37, 49, 50, 61]. Some approaches adopt *central differential privacy* [15, 23, 30, 33, 58], where a trusted curator can access the entire graph. However, if this curator is compromised, all users' privacy is at risk [18, 21]. Another line of research adopts *local differential privacy* [11, 18–20, 28, 35, 41, 48]. Two main paradigms exist for graph analysis under LDP: (1) general-purpose synthetic graph construction [13, 16, 22, 29, 36, 56, 60] and (2) problem-specific algorithmic design. The former often suffers from low data utility due to the loss of graph structure. Under the second category, many works are devoted to motif counting. [18] introduces one-round and two-round algorithms for triangle counting under edge LDP, while [19] improves communication cost and estimation error. [11] offers an in-depth technical analysis of these algorithms. [28] and [41] study triangle counting in the localized setting with extended local views. [27] attempts to improve data utility for triangle counting under edge LDP in a crypto-assisted manner. [41] also addresses three-hop paths and k -cliques on small k values. [20] estimates the 4-cycle and triangle counts under the shuffle model, where users' messages are shuffled before being sent to the data curator. [40] proposes k -star LDP to addresses differentially private (p, q) -biclique counting over bipartite graphs. Specifically, each vertex reports its perturbed k -star neighbor lists instead of the classic edge neighbor lists to the data curator. In addition, [6] studies core decomposition under edge LDP, leading to approximate solutions for densest subgraph discovery. [8] further proves a purely additive loss for the densest subgraph problem under edge LDP. A recent work [30] studies

publishing the histogram of common neighbor counts under the centralized model, which differs from our setting.

7 Conclusion

In this paper, we study the problem of common neighborhood estimation on bipartite graphs under edge LDP. To address overcounting with the Naive approach, we propose the OneR algorithm that leverages the flipping probability to construct unbiased estimates. To improve data utility, we propose a multiple-round framework and a single-source algorithm MultiR-SS, which enables the query vertices to download noisy edges and construct unbiased estimators locally. This significantly reduces error compared to OneR by limiting the candidate pool. To tackle cross-round privacy budget allocation and the variety of query vertices, we propose the MultiR-DS algorithm that returns a weighted average of two unbiased estimators associated with two query vertices. We propose novel optimizations to adjust the privacy budgets of each round and the contribution of each estimator based on the query vertices. Experiments on 15 real-world bipartite graphs validate the effectiveness and efficiency of the proposed multiple round algorithms.

8 Acknowledgments

Kai Wang is the corresponding author. Kai Wang is supported by NSFC 62302294 and U2241211. Wenjie Zhang is supported by ARC DP230101445 and FT210100303. Xuemin Lin is supported by NSFC U2241211. Ying Zhang is supported by ARC DP210101393, LP210301046, and DP230101445. We would like to thank the anonymous reviewers for their valuable and constructive feedback.

References

- [1] Aman Abidi, Lu Chen, Rui Zhou, and Chengfei Liu. 2022. Searching Personalized k k -Wing in Bipartite Graphs. *IEEE Transactions on Knowledge and Data Engineering* 35, 8 (2022), 8515–8528.
- [2] Sinan G Aksoy, Tamara G Kolda, and Ali Pinar. 2017. Measuring and modeling bipartite graphs with community structure. *Journal of Complex Networks* 5, 4 (2017), 581–603.
- [3] Remco R Bouckaert. 2008. Practical bias variance decomposition. In *AI 2008: Advances in Artificial Intelligence: 21st Australasian Joint Conference on Artificial Intelligence Auckland, New Zealand, December 1-5, 2008. Proceedings* 21. Springer, 247–257.
- [4] Xiaoshuang Chen, Kai Wang, Xuemin Lin, Wenjie Zhang, Lu Qin, and Ying Zhang. 2021. Efficiently answering reachability and path queries on temporal bipartite graphs. *Proceedings of the VLDB Endowment* (2021).
- [5] Wei-Yen Day, Ninghui Li, and Min Lyu. 2016. Publishing graph degree distribution with node differential privacy. In *Proceedings of the 2016 International Conference on Management of Data*. 123–138.
- [6] Laxman Dhulipala, Quanquan C Liu, Sofya Raskhodnikova, Jessica Shi, Julian Shun, and Shangdi Yu. 2022. Differential Privacy from Locally Adjustable Graph Algorithms: k -Core Decomposition, Low Out-Degree Ordering, and Densest Subgraphs. In *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 754–765.
- [7] Xiaofeng Ding, Xiaodong Zhang, Zhifeng Bao, and Hai Jin. 2018. Privacy-preserving triangle counting in large graphs. In *Proceedings of the 27th ACM international conference on information and knowledge management*. 1283–1292.
- [8] Michael Dinitz, Satyen Kale, Silvio Lattanzi, and Sergei Vassilvitskii. 2023. Improved Differentially Private Densest Subgraph: Local and Purely Additive. *arXiv preprint arXiv:2308.10316* (2023).
- [9] Zheng Dong, Xin Huang, Guorui Yuan, Hengshu Zhu, and Hui Xiong. 2021. Butterfly-core community search over labeled graphs. *arXiv preprint arXiv:2105.08628* (2021).
- [10] Cynthia Dwork, Aaron Roth, et al. 2014. The algorithmic foundations of differential privacy. *Foundations and Trends® in Theoretical Computer Science* 9, 3–4 (2014), 211–407.
- [11] Talya Eden, Quanquan C Liu, Sofya Raskhodnikova, and Adam Smith. 2023. Triangle Counting with Local Edge Differential Privacy. *arXiv preprint arXiv:2305.02263* (2023).
- [12] Aurel Galántai. 2000. The theory of Newton’s method. *J. Comput. Appl. Math.* 124, 1-2 (2000), 25–44.
- [13] Tianchong Gao, Feng Li, Yu Chen, and XuKai Zou. 2018. Local differential privately anonymizing online social networks under hrg-based model. *IEEE Transactions on Computational Social Systems* 5, 4 (2018), 1009–1020.
- [14] Michael Hay, Chao Li, Jerome Miklau, and David Jensen. 2009. Accurate estimation of the degree distribution of private networks. In *2009 Ninth IEEE International Conference on Data Mining*. IEEE, 169–178.

- [15] Michael Hay, Vibhor Rastogi, Gerome Miklau, and Dan Suciu. 2009. Boosting the accuracy of differentially-private histograms through consistency. *arXiv preprint arXiv:0904.0942* (2009).
- [16] Lihe Hou, Weiwei Ni, Sen Zhang, Nan Fu, and Dongyue Zhang. 2023. PPDU: dynamic graph publication with local differential privacy. *Knowledge and Information Systems* 65, 7 (2023), 2965–2989.
- [17] Zan Huang. 2010. Link prediction based on graph topology: The predictive value of generalized clustering coefficient. *Available at SSRN 1634014* (2010).
- [18] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. 2021. Locally Differentially Private Analysis of Graph Statistics.. In *USENIX Security Symposium*. 983–1000.
- [19] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. 2022. Communication-Efficient Triangle Counting under Local Differential Privacy. In *31st USENIX Security Symposium (USENIX Security 22)*. 537–554.
- [20] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. 2022. Differentially Private Triangle and 4-Cycle Counting in the Shuffle Model. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 1505–1519.
- [21] Honglu Jiang, Jian Pei, Dongxiao Yu, Jiguo Yu, Bei Gong, and Xiuzhen Cheng. 2021. Applications of differential privacy in social network analysis: A survey. *IEEE Transactions on Knowledge and Data Engineering* 35, 1 (2021), 108–127.
- [22] Xin Ju, Xiaofeng Zhang, and William K Cheung. 2019. Generating synthetic graphs for large sensitive and correlated social networks. In *2019 IEEE 35th international conference on data engineering workshops (ICDEW)*. IEEE, 286–293.
- [23] Vishesh Karwa, Sofya Raskhodnikova, Adam Smith, and Grigory Yaroslavtsev. 2011. Private analysis of graph structure. *Proceedings of the VLDB Endowment* 4, 11 (2011), 1146–1157.
- [24] Elizabeth A Leicht, Petter Holme, and Mark EJ Newman. 2006. Vertex similarity in networks. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 73, 2 (2006), 026120.
- [25] Zhao Li, Xin Shen, Yuhang Jiao, Xuming Pan, Pengcheng Zou, Xianling Meng, Chengwei Yao, and Jiajun Bu. 2020. Hierarchical bipartite graph neural networks: Towards large-scale e-commerce applications. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 1677–1688.
- [26] Wanyu Lin, Baochun Li, and Cong Wang. 2022. Towards private learning on decentralized graphs with local differential privacy. *IEEE Transactions on Information Forensics and Security* 17 (2022), 2936–2946.
- [27] Shang Liu, Yang Cao, Takao Murakami, Jinfei Liu, and Masatoshi Yoshikawa. 2023. CARGO: Crypto-Assisted Differentially Private Triangle Counting without Trusted Servers. *arXiv preprint arXiv:2312.12938* (2023).
- [28] Yuhan Liu, Suyun Zhao, Yixuan Liu, Dan Zhao, Hong Chen, and Cuiping Li. 2022. Collecting triangle counts with edge relationship local differential privacy. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 2008–2020.
- [29] Zichun Liu, Liusheng Huang, Hongli Xu, Wei Yang, and Shaowei Wang. 2020. PrivAG: Analyzing attributed graph data with local differential privacy. In *2020 IEEE 26th International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE, 422–429.
- [30] Chaojie Lv, Xiaokui Xiao, Lan Zhang, and Ting Yu. 2024. Publishing Common Neighbors Histograms of Social Networks under Edge Differential Privacy. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*. 1099–1113.
- [31] Tianzi Lv, Huanzhou Li, Zhangguo Tang, Fangzhou Fu, Jian Cao, and Jian Zhang. 2021. Publishing Triangle Counting Histogram in Social Networks Based on Differential Privacy. *Security and Communication Networks* 2021 (2021), 1–16.
- [32] Kamalkumar R Macwan and Sankita J Patel. 2018. Node differential privacy in social graph degree publishing. *Procedia computer science* 143 (2018), 786–793.
- [33] Kobbi Nissim, Sofya Raskhodnikova, and Adam Smith. 2007. Smooth sensitivity and sampling in private data analysis. In *Proceedings of the thirty-ninth annual ACM symposium on Theory of computing*. 75–84.
- [34] Clare M O'Connor, Jill U Adams, and Jennifer Fairman. 2010. Essentials of cell biology. *Cambridge, MA: NPG Education* 1 (2010), 54.
- [35] Zhan Qin, Ting Yu, Yin Yang, Issa Khalil, Xiaokui Xiao, and Kui Ren. [n. d.]. Generating synthetic decentralized social graphs with local differential privacy. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security* (2017). 425–438.
- [36] Zhan Qin, Ting Yu, Yin Yang, Issa Khalil, Xiaokui Xiao, and Kui Ren. 2017. Generating synthetic decentralized social graphs with local differential privacy. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 425–438.
- [37] Sina Sajadmanesh and Daniel Gatica-Perez. 2021. Locally private graph neural networks. In *Proceedings of the 2021 ACM SIGSAC conference on computer and communications security*. 2130–2145.
- [38] John G Saw, Mark CK Yang, and Tse Chin Mo. 1984. Chebyshev inequality with estimated mean and variance. *The American Statistician* 38, 2 (1984), 130–132.
- [39] Marija Stankova, Stiene Praet, David Martens, and Foster Provost. 2021. Node classification over bipartite graphs through projection. *Machine Learning* 110 (2021), 37–87.

- [40] Henan Sun, Zhengyu Wu, Rong-Hua Li, Guoren Wang, and Zening Li. 2024. K-stars LDP: A Novel Framework for (p, q)-clique Enumeration under Local Differential Privacy. *arXiv preprint arXiv:2403.01788* (2024).
- [41] Haipai Sun, Xiaokui Xiao, Issa Khalil, Yin Yang, Zhan Qin, Hui Wang, and Ting Yu. 2019. Analyzing subgraph statistics from extended local views with decentralized differential privacy. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 703–717.
- [42] Jimeng Sun, Huiming Qu, Deepayan Chakrabarti, and Christos Faloutsos. 2005. Neighborhood formation and anomaly detection in bipartite graphs. In *Fifth IEEE international conference on data mining (ICDM'05)*. IEEE, 8–pp.
- [43] Charalampos E Tsourakakis. 2014. Toward quantifying vertex similarity in networks. *Internet Mathematics* 10, 3-4 (2014), 263–286.
- [44] Jun Wang, Arjen P De Vries, and Marcel JT Reinders. 2006. Unifying user-based and item-based collaborative filtering approaches by similarity fusion. In *Proceedings of the 29th annual international ACM SIGIR conference on Research and development in information retrieval*. ACM, 501–508.
- [45] Jianwei Wang, Kai Wang, Xuemin Lin, Wenjie Zhang, and Ying Zhang. 2024. Efficient Unsupervised Community Search with Pre-trained Graph Transformer. *Proc. VLDB Endow.* 17, 9 (2024), 2227–2240. <https://www.vldb.org/pvldb/vol17/p2227-wang.pdf>
- [46] Zhibin Wang, Longbin Lai, Yixue Liu, Bing Shui, Chen Tian, and Sheng Zhong. 2023. I/O-Efficient Butterfly Counting at Scale. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–27.
- [47] Stanley L Warner. 1965. Randomized response: A survey technique for eliminating evasive answer bias. *J. Amer. Statist. Assoc.* 60, 309 (1965), 63–69.
- [48] Chengkun Wei, Shouling Ji, Changchang Liu, Wenzhi Chen, and Ting Wang. 2020. AsgLDP: Collecting and generating decentralized attributed graphs with local differential privacy. *IEEE Transactions on Information Forensics and Security* 15 (2020), 3239–3254.
- [49] Yuecen Wei, Haonan Yuan, Xingcheng Fu, Qingyun Sun, Hao Peng, Xianxian Li, and Chunming Hu. 2024. Poincaré Differential Privacy for Hierarchy-aware Graph Embedding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 9160–9168.
- [50] Fan Wu, Yunhui Long, Ce Zhang, and Bo Li. 2022. Linkteller: Recovering private edges from graph neural networks via influence analysis. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2005–2024.
- [51] Qingyu Xu, Feng Zhang, Zhiming Yao, Lv Lu, Xiaoyong Du, Dong Deng, and Bingsheng He. 2022. Efficient load-balanced butterfly counting on GPU. *Proceedings of the VLDB Endowment* 15, 11 (2022), 2450–2462.
- [52] Jianye Yang, Yun Peng, Dian Ouyang, Wenjie Zhang, Xuemin Lin, and Xiang Zhao. 2023. (p, q)-biclique counting and enumeration for large sparse bipartite graphs. *The VLDB Journal* (2023), 1–25.
- [53] Jianye Yang, Yun Peng, and Wenjie Zhang. 2021. (p, q)-biclique counting and enumeration for large sparse bipartite graphs. *Proceedings of the VLDB Endowment* 15, 2 (2021), 141–153.
- [54] Mengmeng Yang, Taolin Guo, Tianqing Zhu, Ivan Tjuawinata, Jun Zhao, and Kwok-Yan Lam. 2023. Local differential privacy and its applications: A comprehensive survey. *Computer Standards & Interfaces* (2023), 103827.
- [55] Renchi Yang. 2022. Efficient and Effective Similarity Search over Bipartite Graphs. In *Proceedings of the ACM Web Conference 2022*. 308–318.
- [56] Qingqing Ye, Haibo Hu, Man Ho Au, Xiaofeng Meng, and Xiaokui Xiao. 2020. LF-GDPR: A framework for estimating graph metrics with local differential privacy. *IEEE Transactions on Knowledge and Data Engineering* 34, 10 (2020), 4905–4920.
- [57] Xiaowei Ye, Rong-Hua Li, Qiangqiang Dai, Hongchao Qin, and Guoren Wang. 2023. Efficient Biclique Counting in Large Bipartite Graphs. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–26.
- [58] Jun Zhang, Graham Cormode, Cecilia M Procopiuc, Divesh Srivastava, and Xiaokui Xiao. 2015. Private release of graph statistics using ladder functions. In *Proceedings of the 2015 ACM SIGMOD international conference on management of data*. 731–745.
- [59] Xianhang Zhang, Hanchen Wang, Jianke Yu, Chen Chen, Xiaoyang Wang, and Wenjie Zhang. 2023. Bipartite graph capsule network. *World Wide Web* 26, 1 (2023), 421–440.
- [60] Yuxuan Zhang, Jianghong Wei, Xiaojian Zhang, Xuexian Hu, and Wenfen Liu. 2018. A two-phase algorithm for generating synthetic graph under local differential privacy. In *Proceedings of the 8th International Conference on Communication and Network Security*. 84–89.
- [61] Xiaochen Zhu, Vincent YF Tan, and Xiaokui Xiao. 2023. Blink: Link Local Differential Privacy in Graph Neural Networks via Bayesian Estimation. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2651–2664.

Received April 2024; revised July 2024; accepted August 2024